Open-Source Lab 2.
In-Memory Database

March 31, 2026

T.A. Yongmin Lee
Dept. of Software
Dankook University

ym.lee@dankook.ac.kr

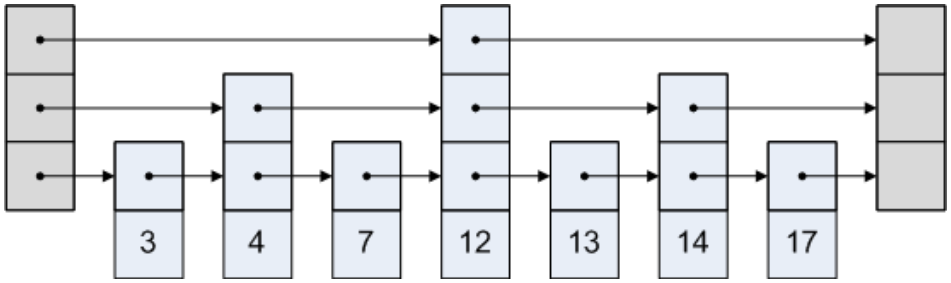
(본 교재는 2026년도 과학기술정보통신부 및 정보통신기획평가원의 'SW중심대학사업' 지원을 받아 제작 되었습니다.)

Contents

- Goal of the assignment
  - To understand RocksDB's in-memory structure
- Skip list implementation
- Bonus: B+Tree implementation
- Code Structure
- How to Submit
  - Via Google Form

Goal of the assignment

- Understand RocksDB's in-memory structure and B+Tree
- Skip list vs. B+Tree



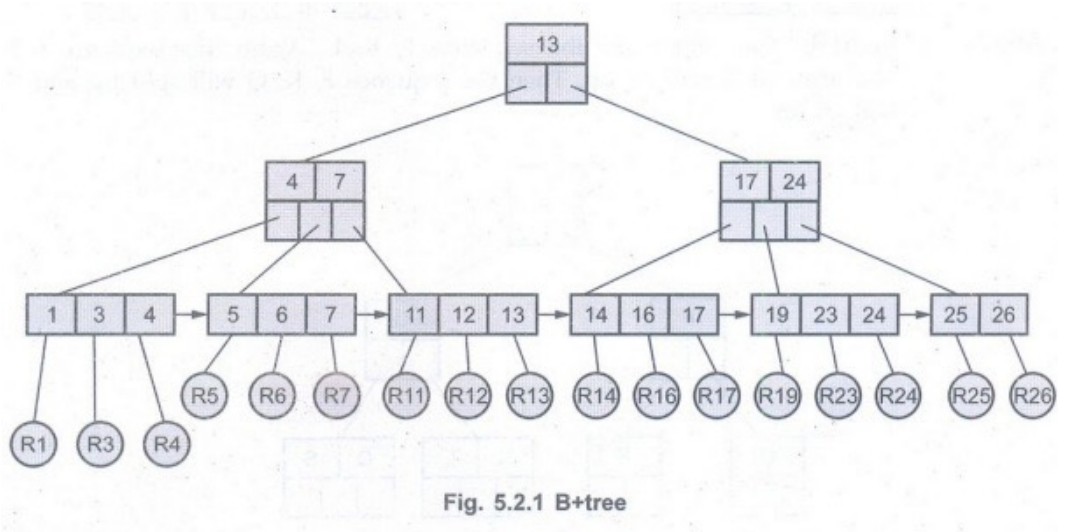


Fig. 5.2.1 B+tree

Ref: https://igoro.com/archive/skip-lists-are-fascinating/ & https://cse.poriyaan.in/topic/b--tree-50544/

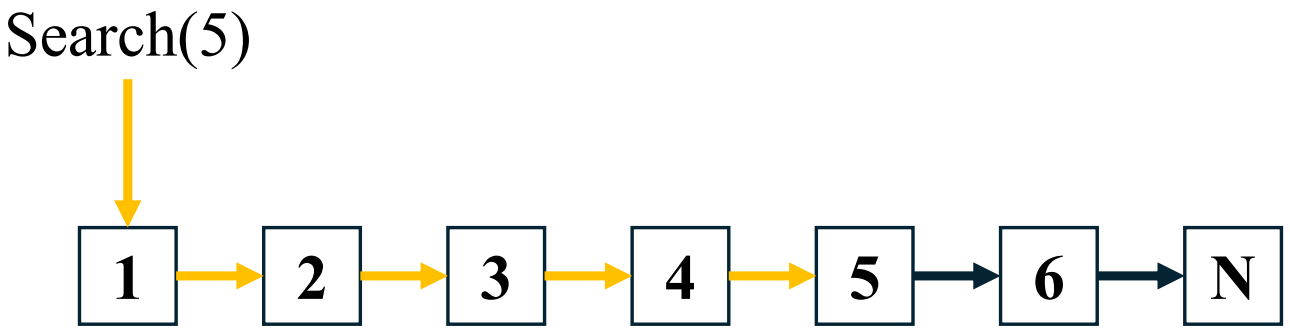
Skip list implementation

- Skip list is a linked list-based data structure
- It is a probabilistic balanced data structure designed to perform search, insertion, and deletion with O(log n) average time complexity using multiple levels
- The goal is to understand and implement it

File Name	Description
Makefile	File for compiling the source code
skiplist.cc & skiplist.h	**Skip list code to be implemented**
memdb.cc & memdb.h	**In-memory DB code to be implemented**
memdb_bench.cc	Benchmark tool code
memdb_test.cc	Skip list testing code

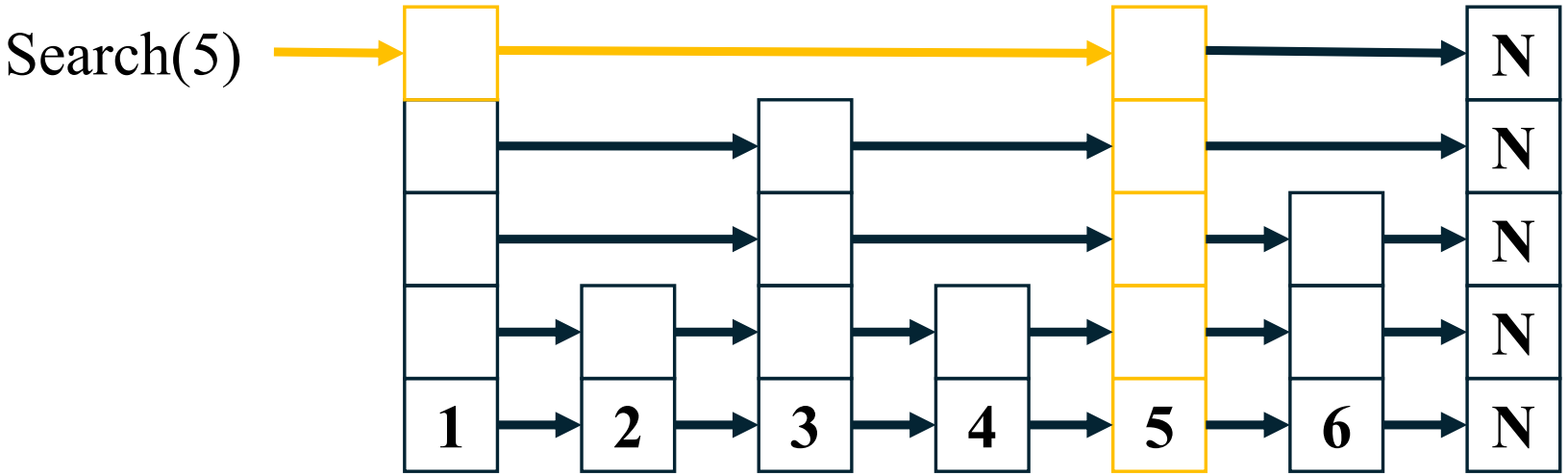
Skip list implementation

- A linked list is a sequence of nodes that contain two fields
  - Data and a link to the next node
- In case of sorted linked list, Search(5) needs 4 steps:



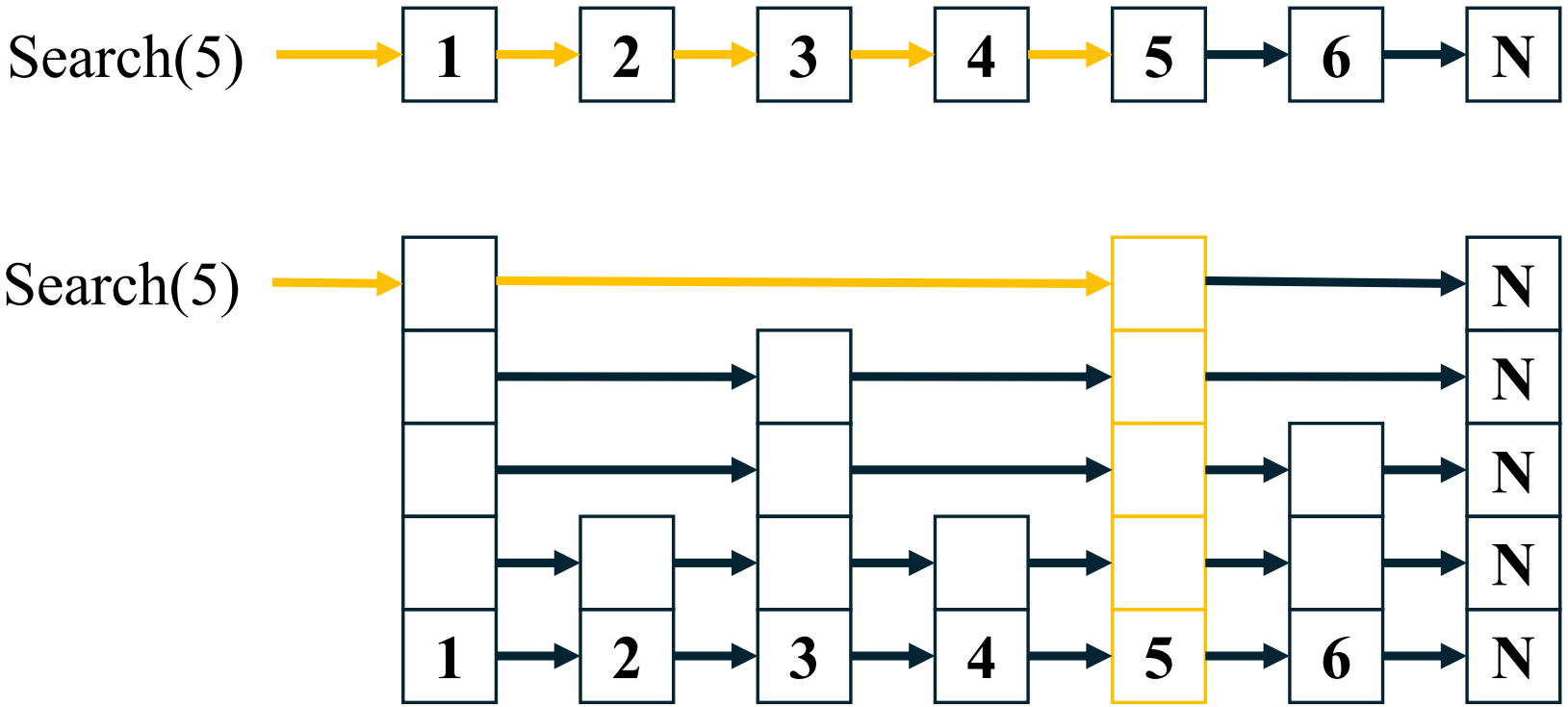
Skip list implementation

- A skip list has multiple levels (layers)
  - The bottom layer 1is an ordinary ordered linked list
  - Element in level i appears in level i + 1 with some fixed probability ***p*** (two commonly used values for p are 1 / 2 or 1 / 4)



Skip list implementation

- A skip list requires less steps compare to linked list



Skip list implementation

- Code

```cpp
#include "skiplist.h"

#include <algorithm>
#include <limits>
#include <random>

// SkipList Constructor. head node, level에 따른 초기 설정 필요
SkipList::SkipList(int max_level, float p)
    : head_(nullptr), max_level_(std::max(1, max_level)), p_(p), next_seq_(1) {
  // code
}

// SkipList Destructor. 생성한 노드에 대해 모두 delete
SkipList::~SkipList() {
  // code
}

// SkipList Put operation시 높이 설정 함수
int SkipList::RandomLevel() {

  // code

  return 0;
}

// SkipList에 새로운 key 및 value를 삽입하는 Put 함수
void SkipList::Put(int key, const std::string& value) {
  // code
}
```

```cpp
// SkipList에 서 key에 해당하는 value 찾기. 존재하면 true, 없으면 (tombstone
// 고려) false 반환. value는 out_value에 저장
bool SkipList::Get(int key, std::string* out_value) const {

  // code

  return false;
}

// SkipList Delete operation. Tombstone으로 삭제 진행
bool SkipList::Delete(int key) {

  // code

  return false;
}

// SkipList range scan operation. 해당하는 노드를 vector에 모아 반환
std::vector<std::pair<int, std::string>>
SkipList::RangeScan(int start_key, int end_key) const {
  std::vector<std::pair<int, std::string>> out;

  // code

  return out;
}
```

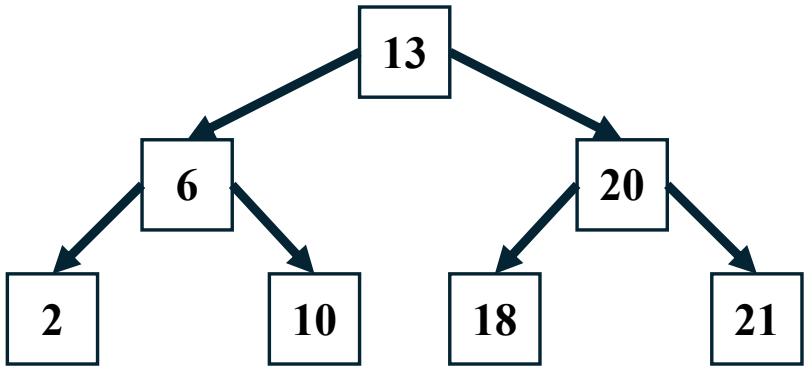
Bonus: B+Tree implementation

- B+ tree is a tree-based data structure derived from B-trees
- It is a self-balancing data structure designed to efficiently perform search, insertion, and deletion operations with O(log n) time complexity by organizing data into multiple sorted levels of internal and leaf nodes.
- The goal is to understand and implement it

File Name	Description
Makefile	File for compiling the source code
bptree.cc & bptree.h	**B+Tree code to be implemented**
memdb.cc & memdb.h	**In-memory DB code to be implemented**
memdb_bench.cc	Benchmark tool code
memdb_test.cc	B+tree testing code

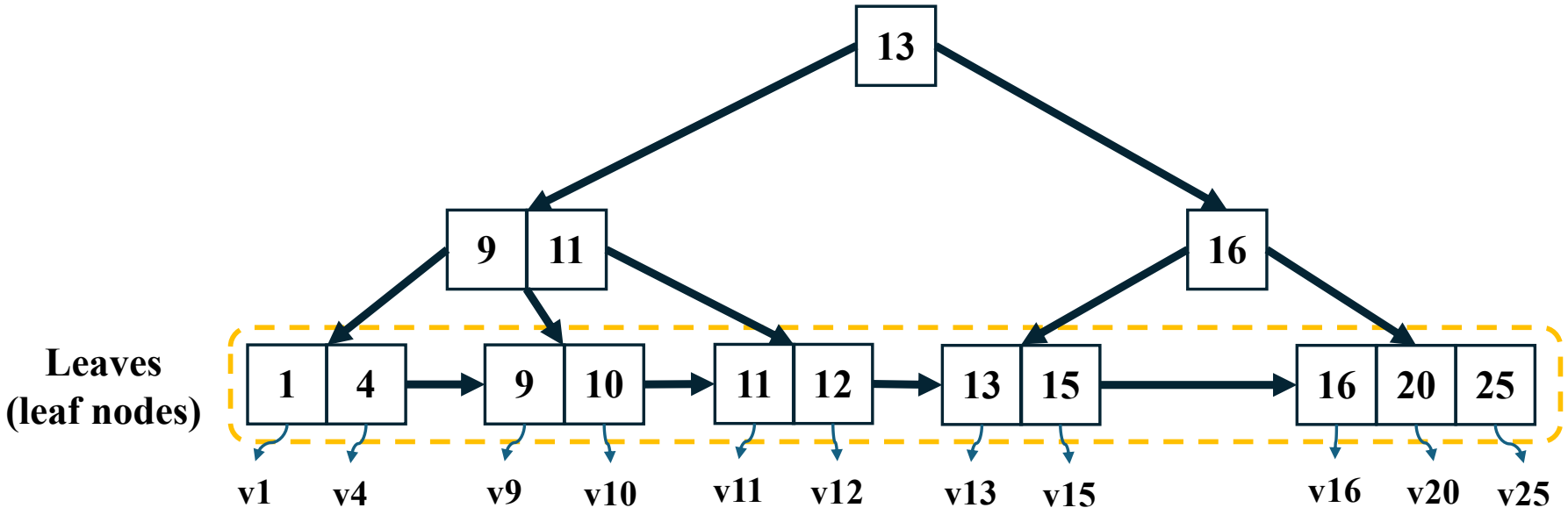
Bonus: B+Tree implementation

- A **binary search tree** consists of connected nodes
  - Any keys to the **left** of a node are **less** than that key
  - Any keys to the **right** of a node are **greater** than that key



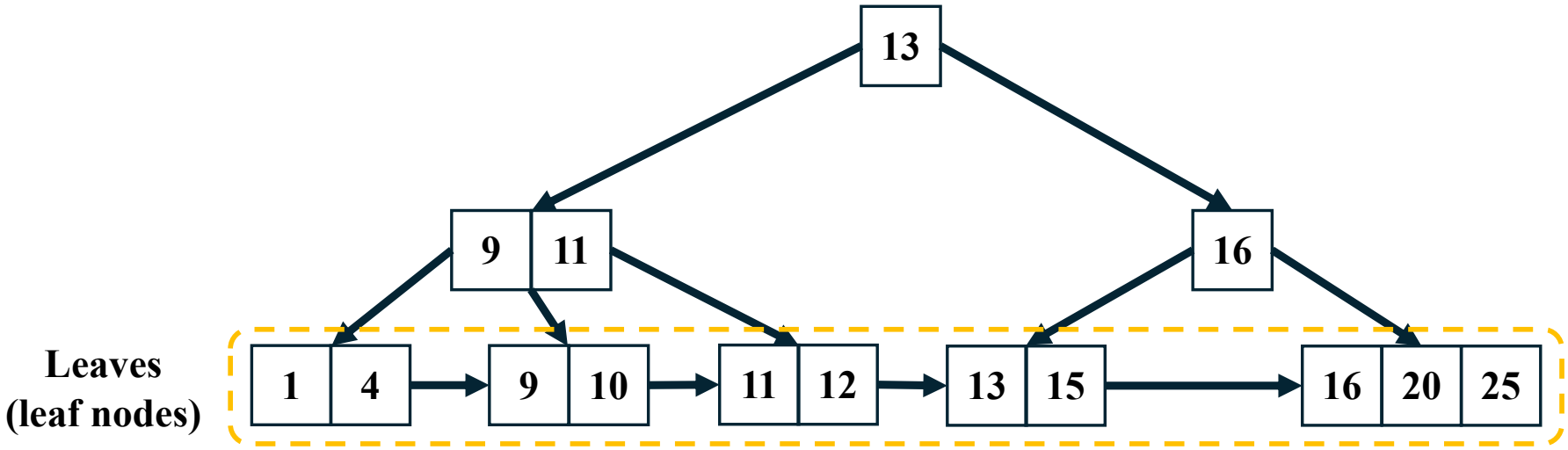
Bonus: B+Tree implementation

- In a **B+tree**, all the data records are stored in the leaf nodes, while the internal nodes only contain keys to guide the search process



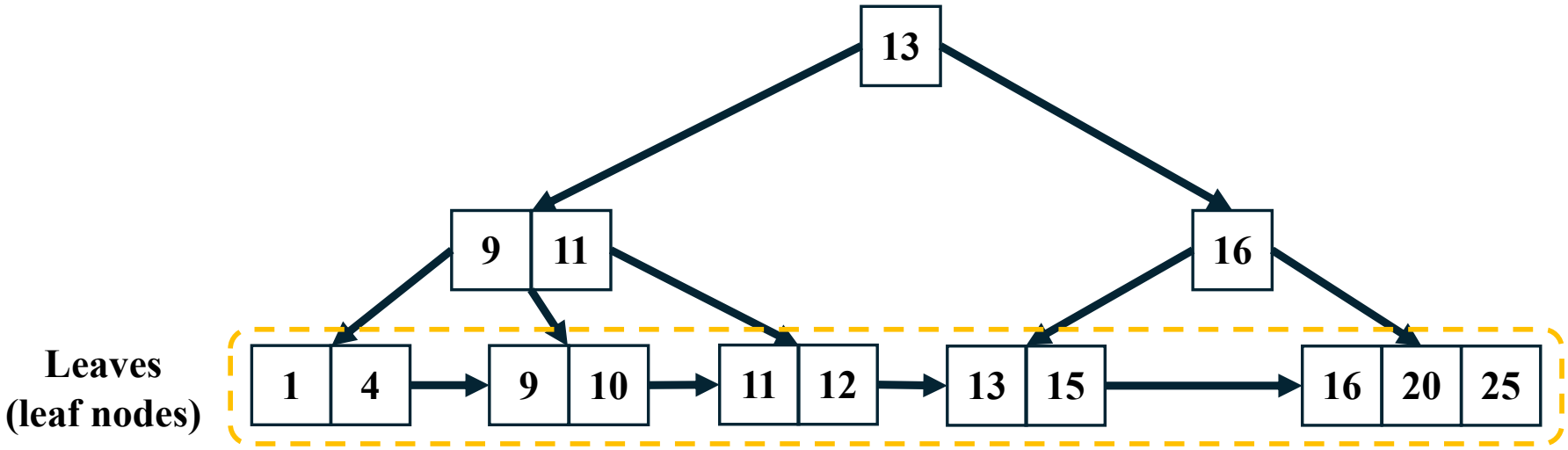
Bonus: B+Tree implementation

- Invariants of B+Trees
  - All leaves are at the same distance from the root
  - The root has at least two children



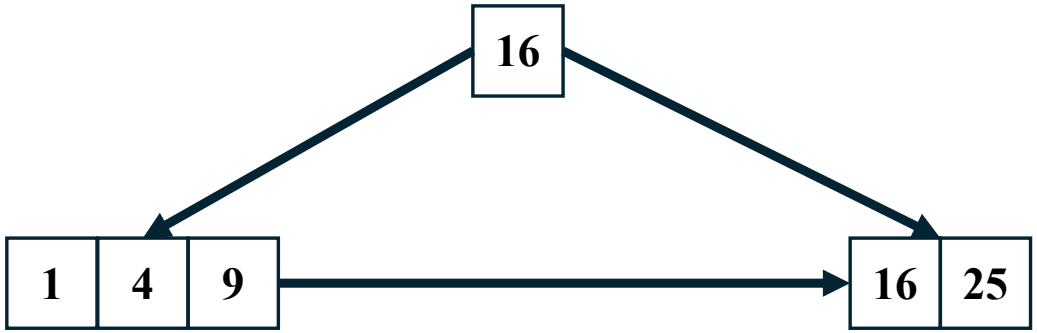
Bonus: B+Tree implementation

- Invariants of B+Trees (M = 4 B-Tree)
  - **M (= 4):** Maximum number of children per node
  - **M − 1 (= 3):** Maximum number of keys per node
  - **⌈M / 2⌉ (= 2):** Minimum number of children per node
  - **⌈M / 2⌉ (= 2):** Minimum number of keys per leaf nodes



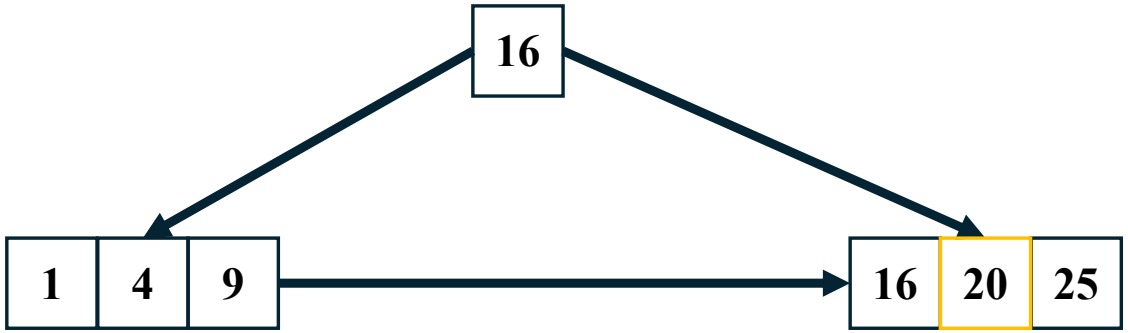
Bonus: B+Tree implementation

- B+Tree Example (M = 4): insertion



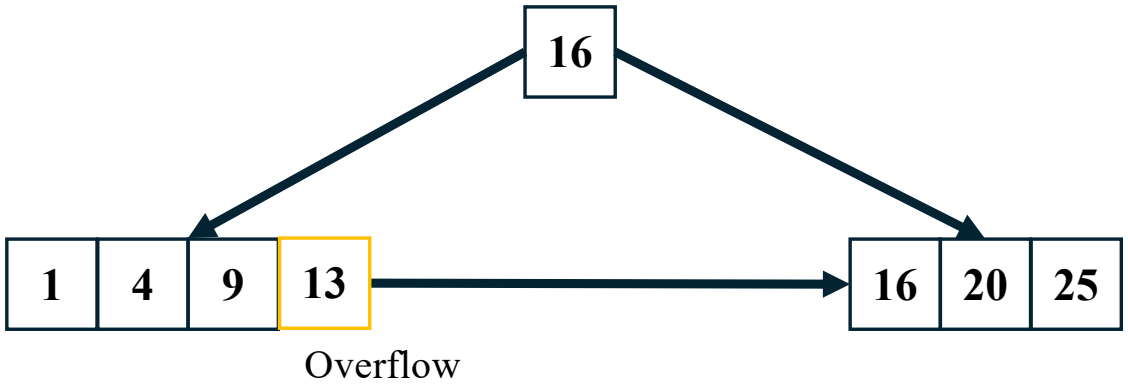
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Insert(20)
  - Find a location in leaf node



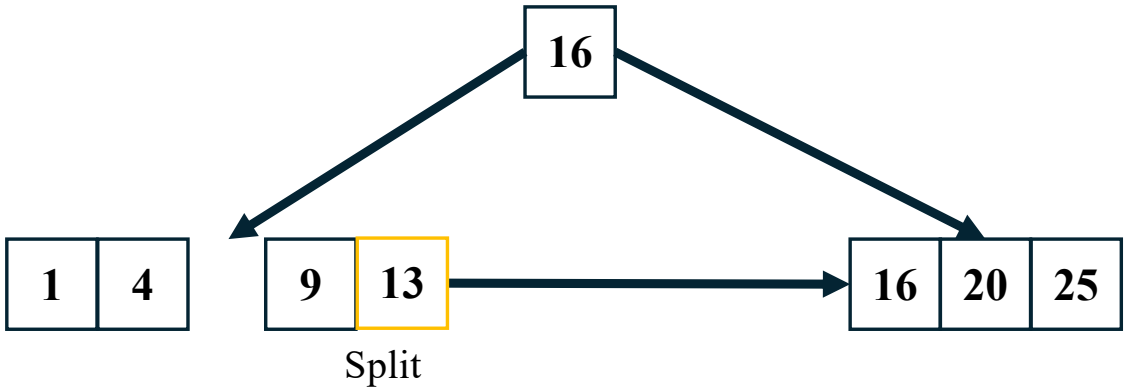
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Insert(13)
  - **M − 1 (= 3):** Maximum number of keys per node



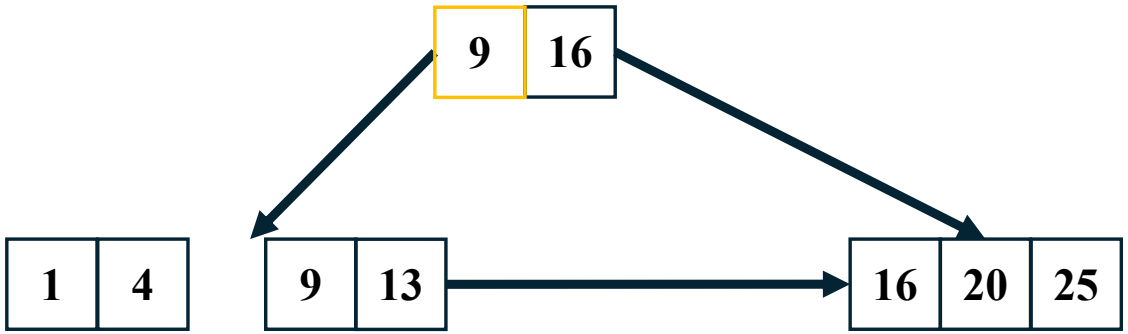
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Insert(13)
  - If full, split the node



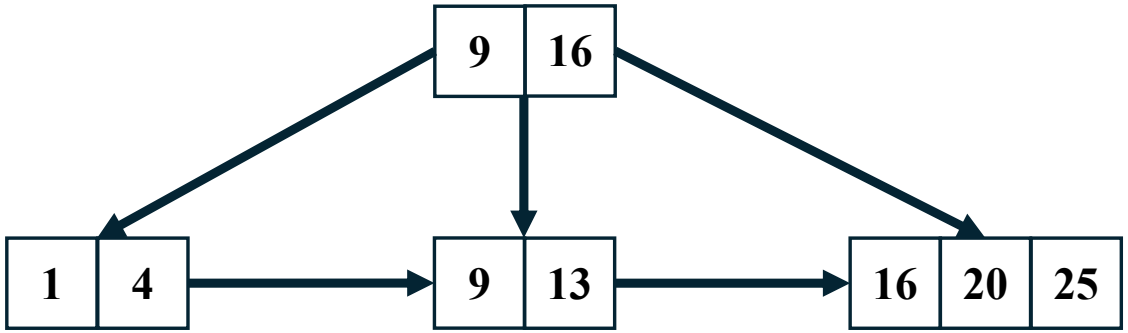
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Insert(13)
  - If the node is a leaf, take a copy of the min. value in the second one



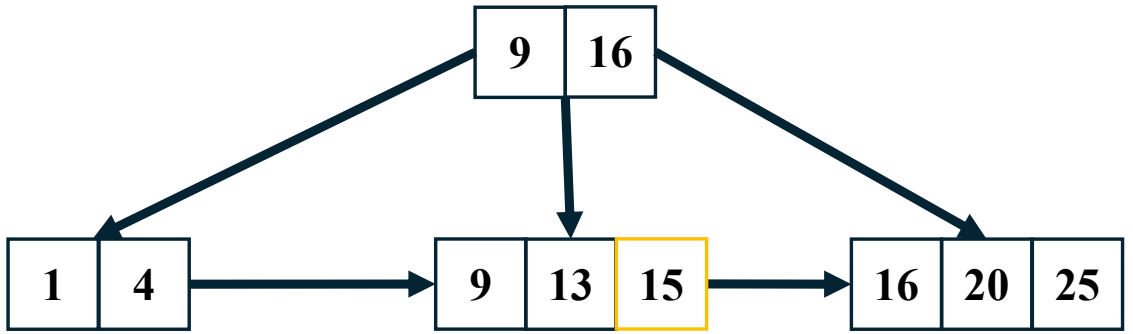
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Insert(13)



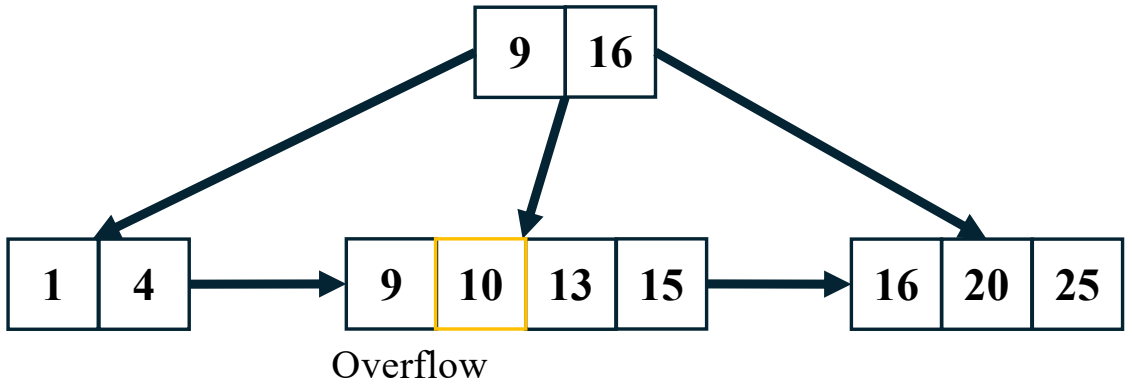
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Insert(15)



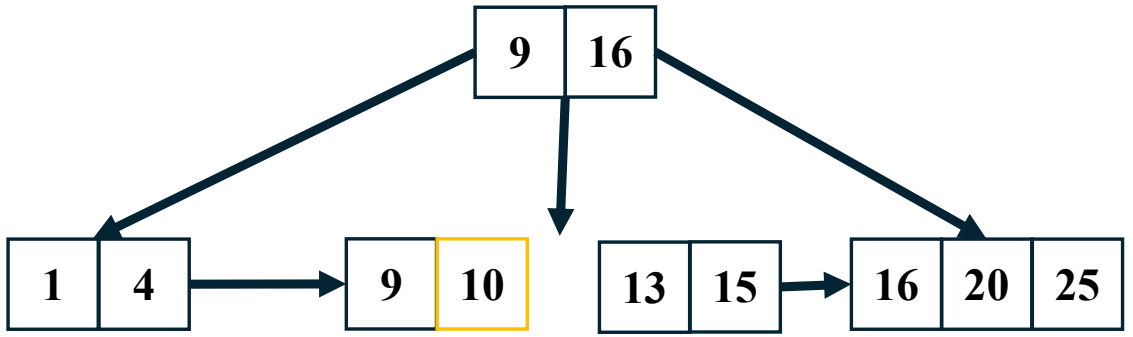
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Insert(10)



Bonus: B+Tree implementation

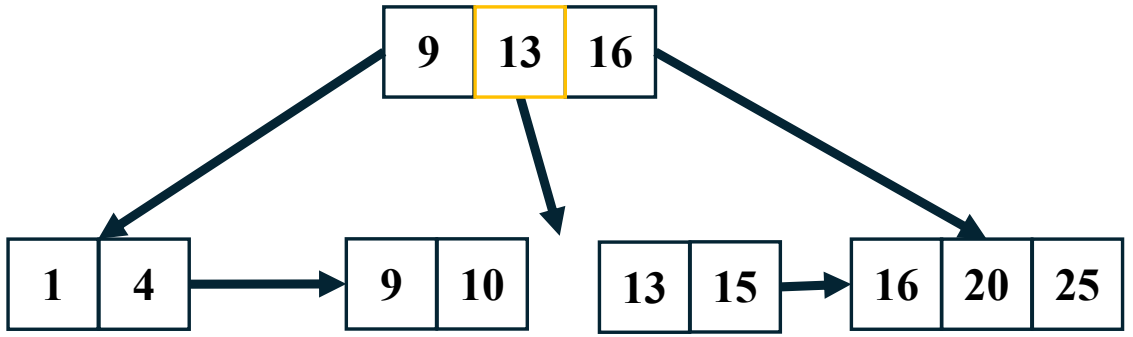
- B+Tree Example (M = 4): Insert(10)



Split

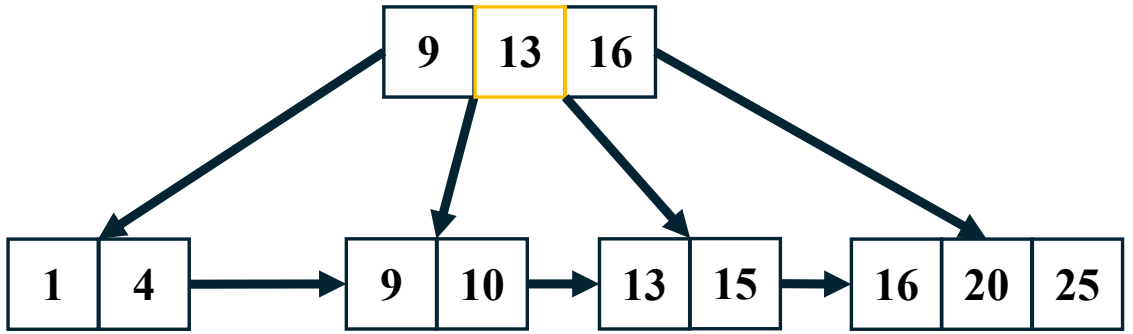
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Insert(10)



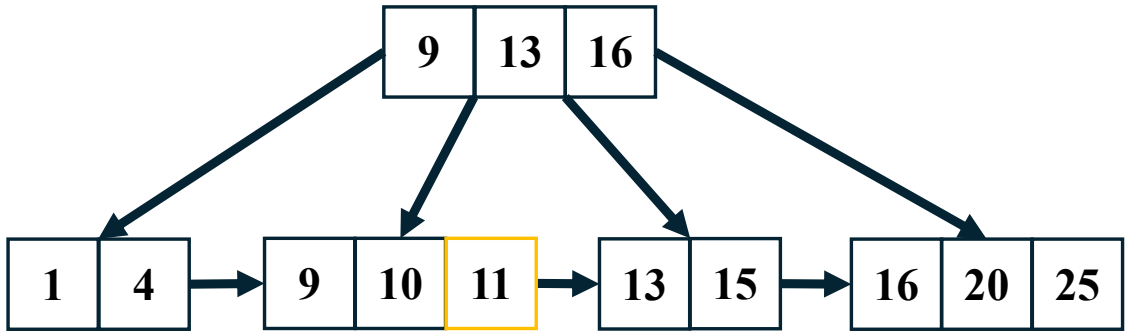
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Insert(10)



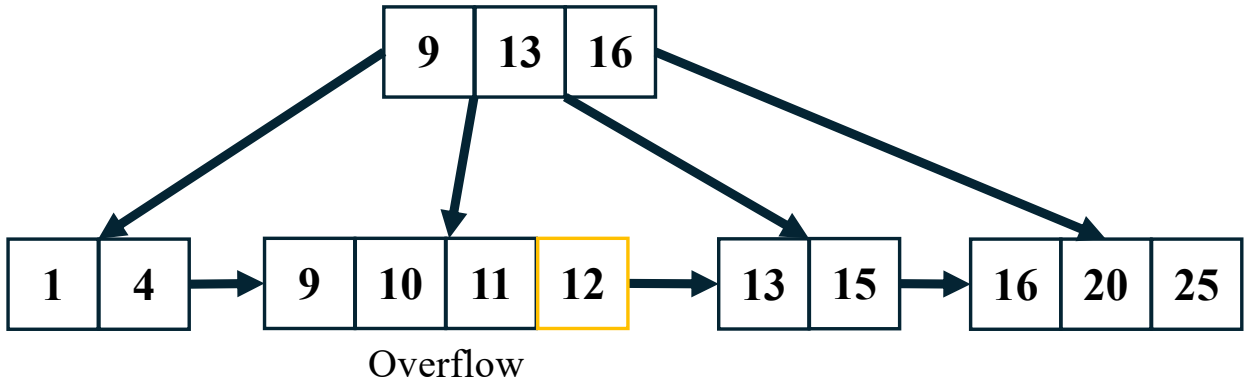
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Insert(11)



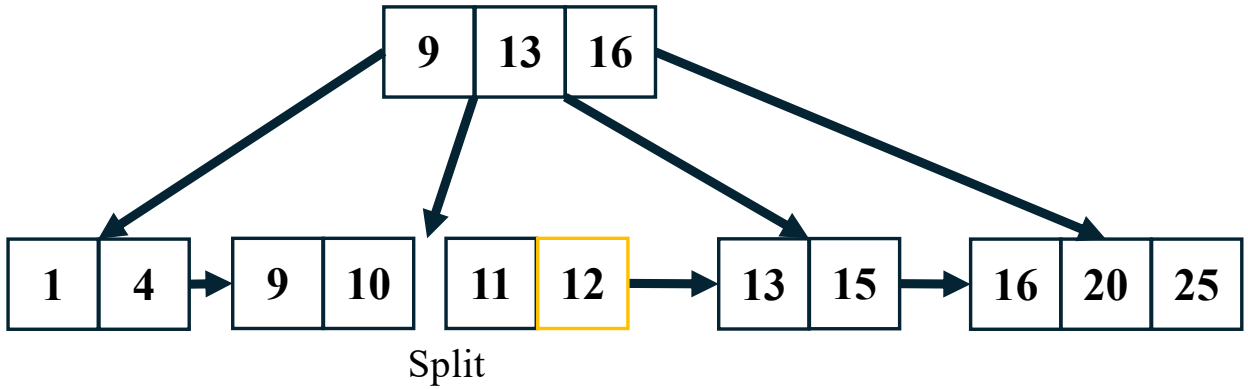
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Insert(12)



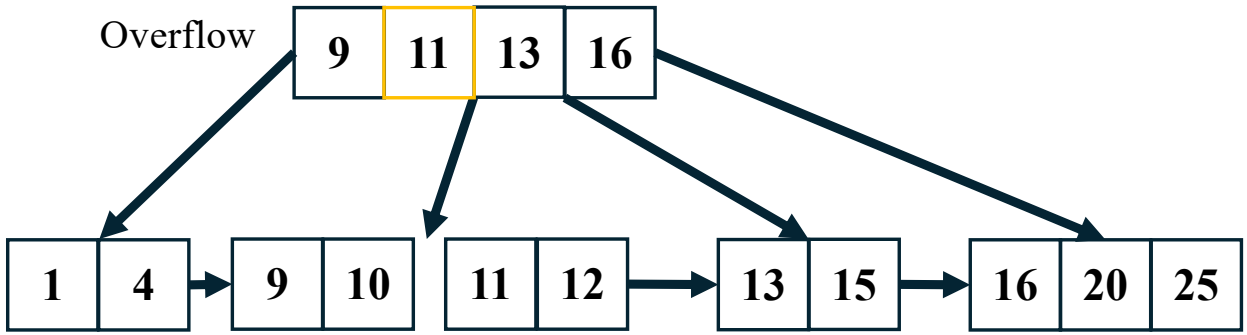
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Insert(12)



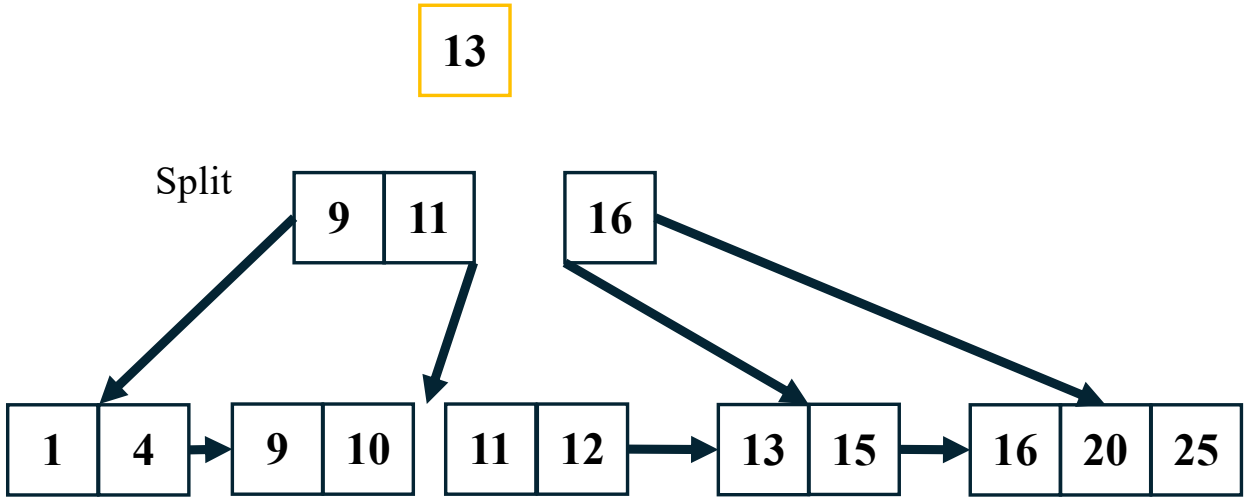
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Insert(12)



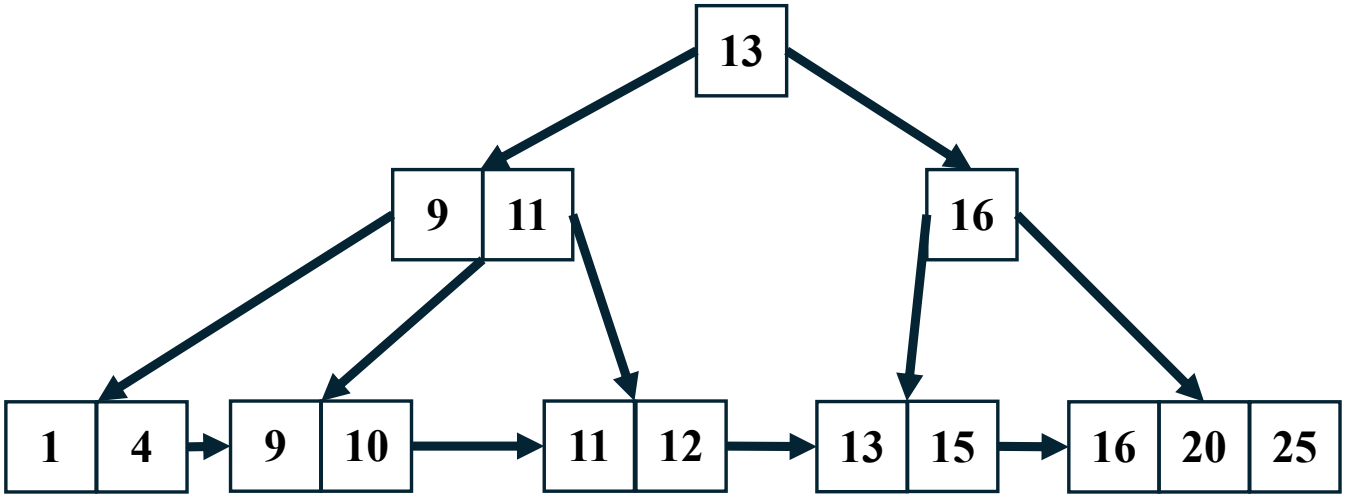
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Insert(12)
  - If the node is a non-leaf, exclude the middle value during the split and repeat this insertion algorithm to insert this excluded value into the parent node.



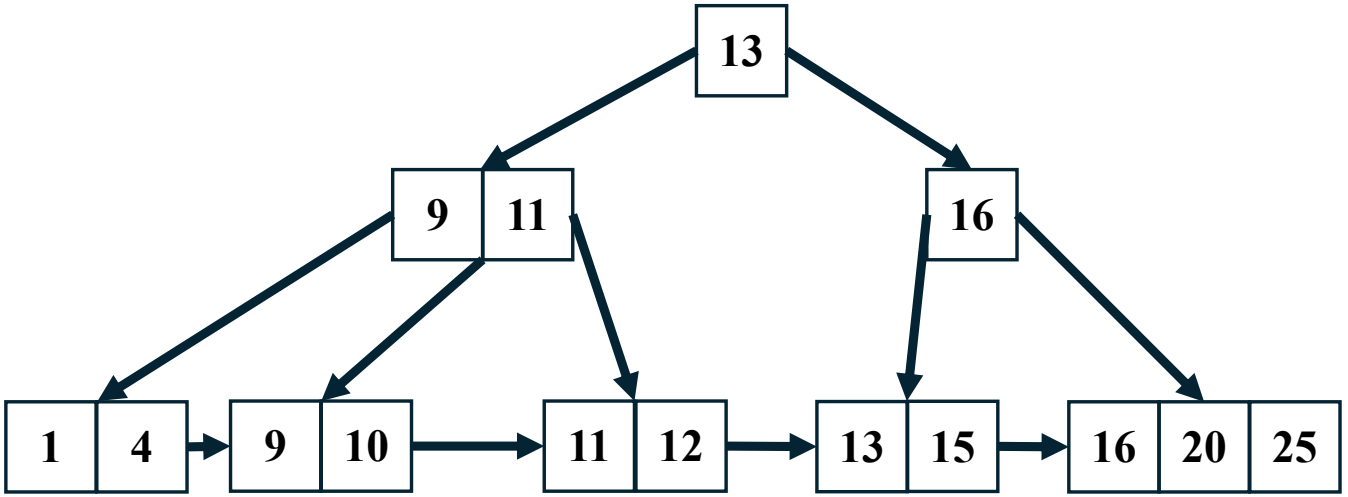
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Insert(12)
  - If the node is a non-leaf, exclude the middle value during the split and repeat this insertion algorithm to insert this excluded value into the parent node.



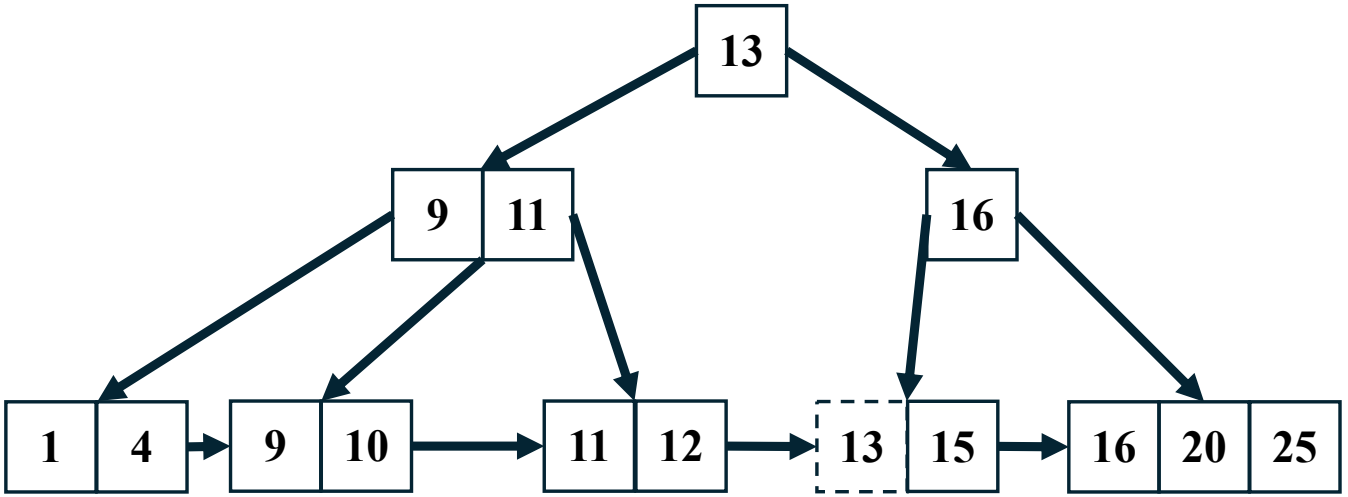
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Deletion



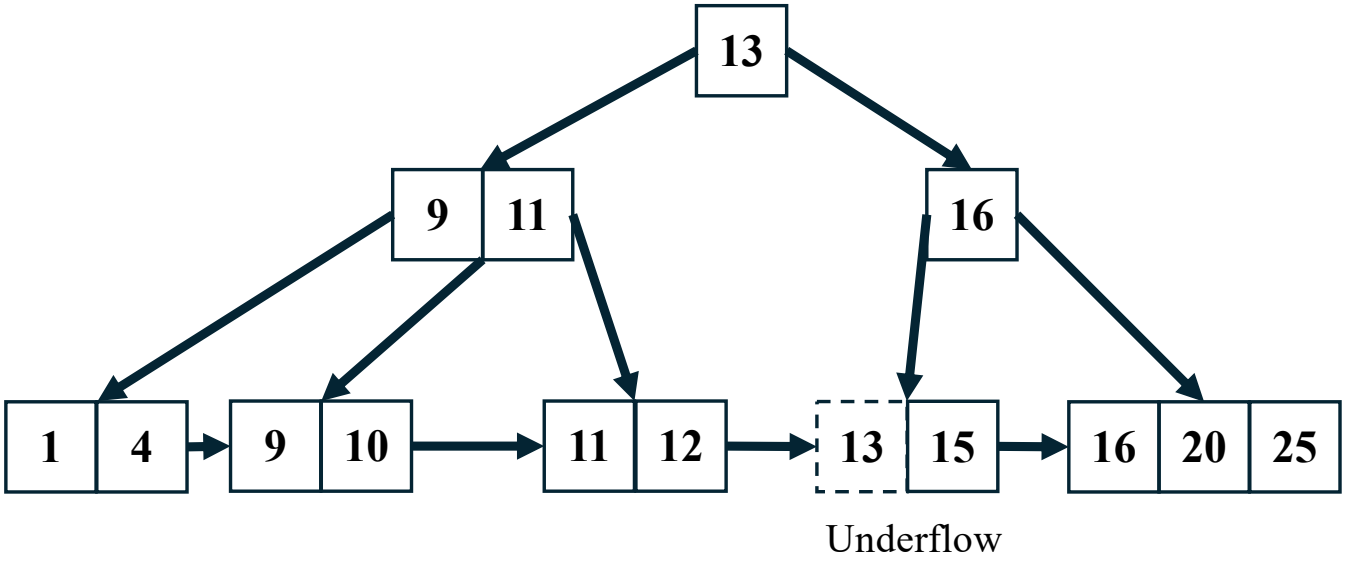
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Delete(13)



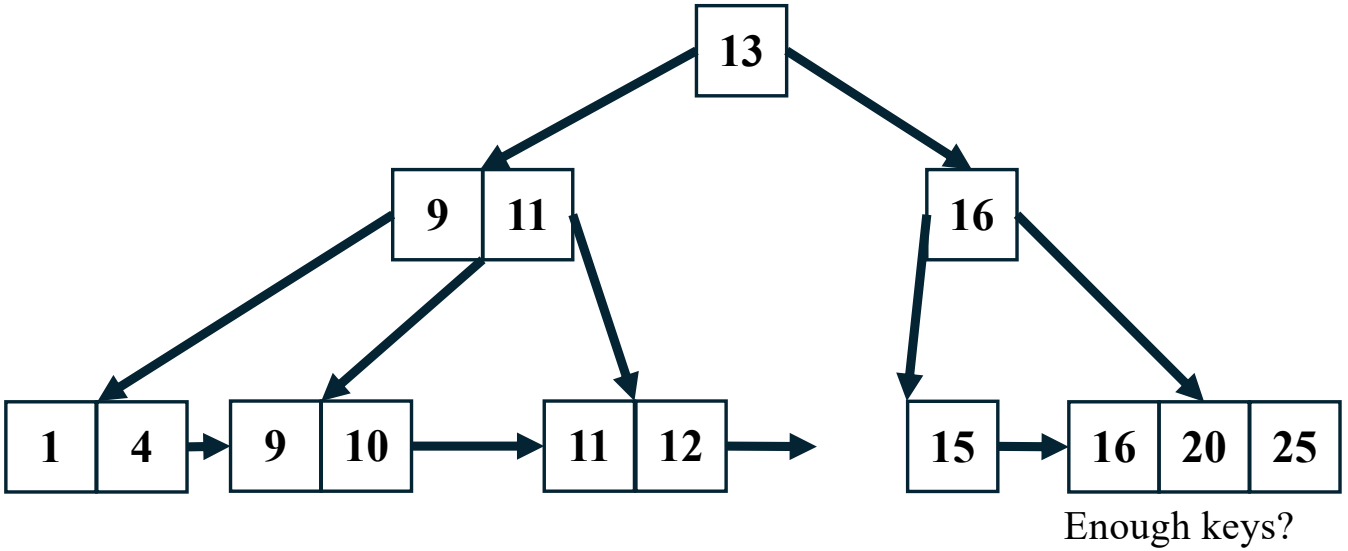
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Delete(13)
  - **[M / 2] (= 2):** Minimum number of keys per leaf nodes



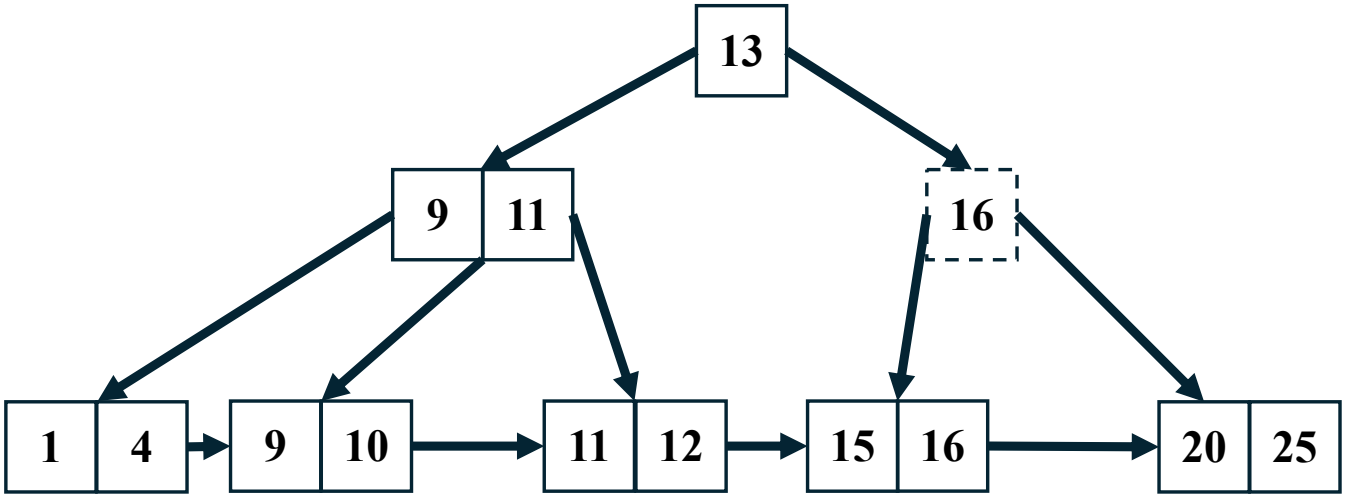
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Delete(13)
  - Ask your close siblings for more keys
  - Checks they have enough keys to give (at least **⌊M / 2⌋ (= 2)** keys)



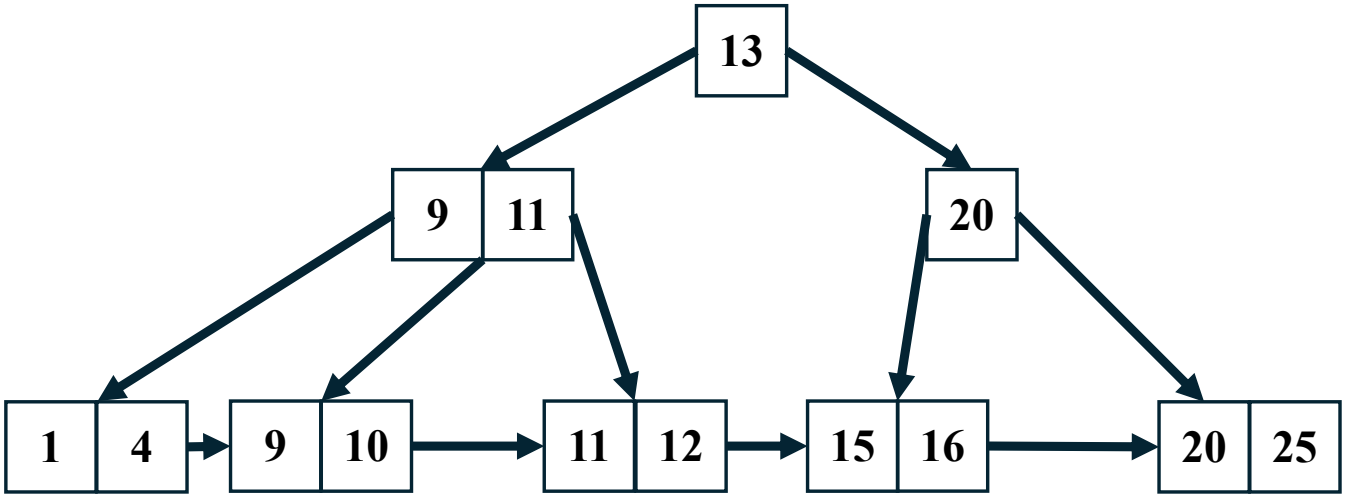
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Delete(13)
  - Change the key of the parent if it has a different split point



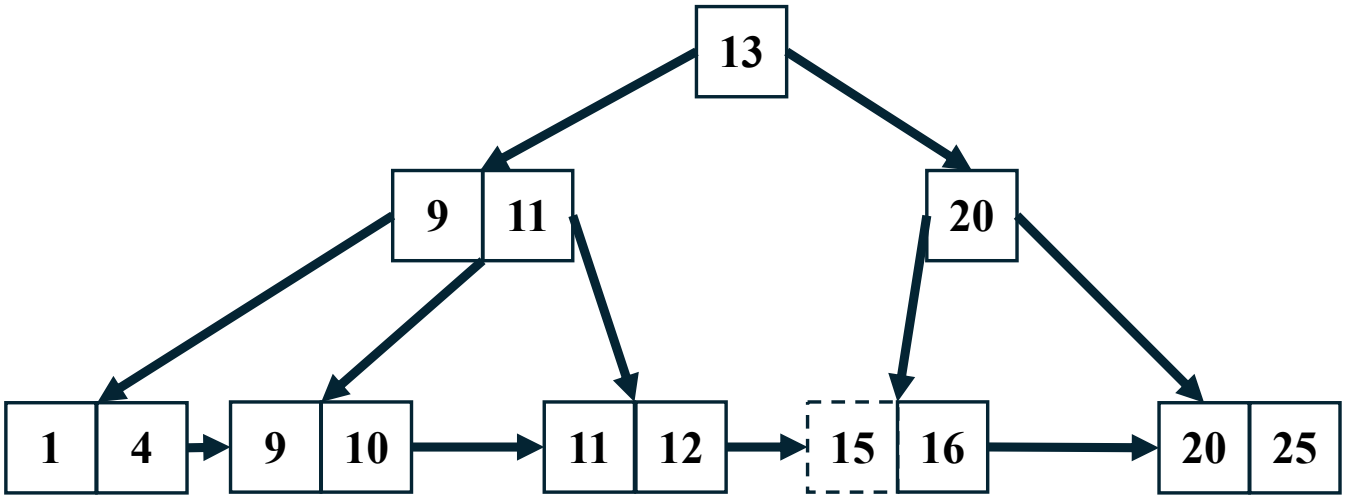
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Delete(13)
  - Change the key of the parent if it has a different split point



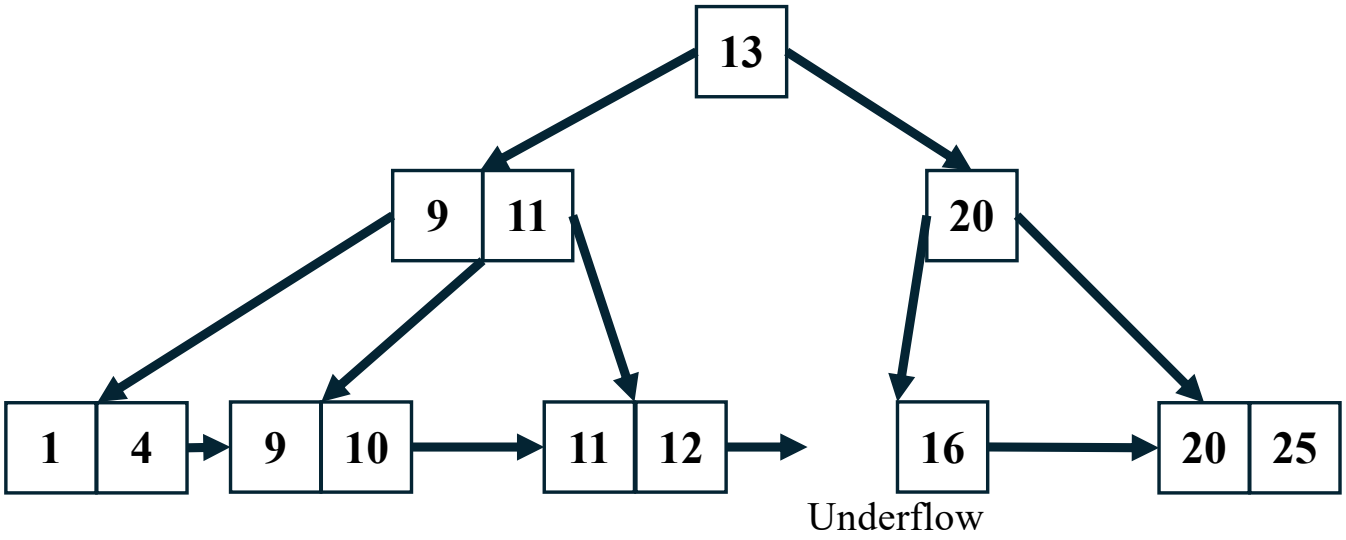
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Delete(15)



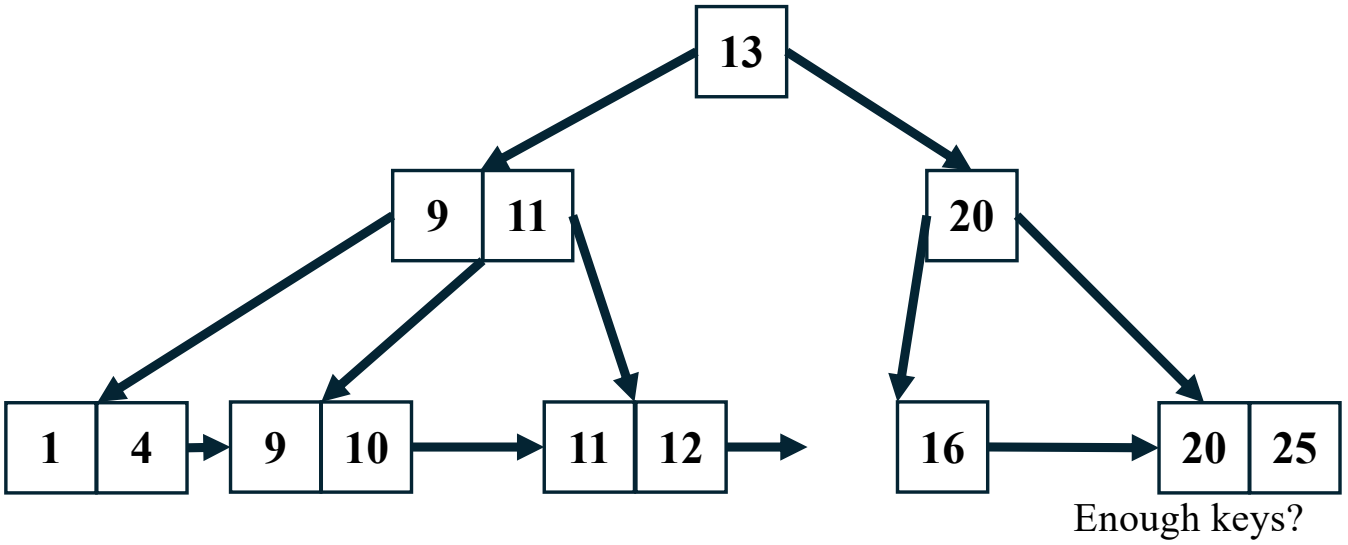
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Delete(15)



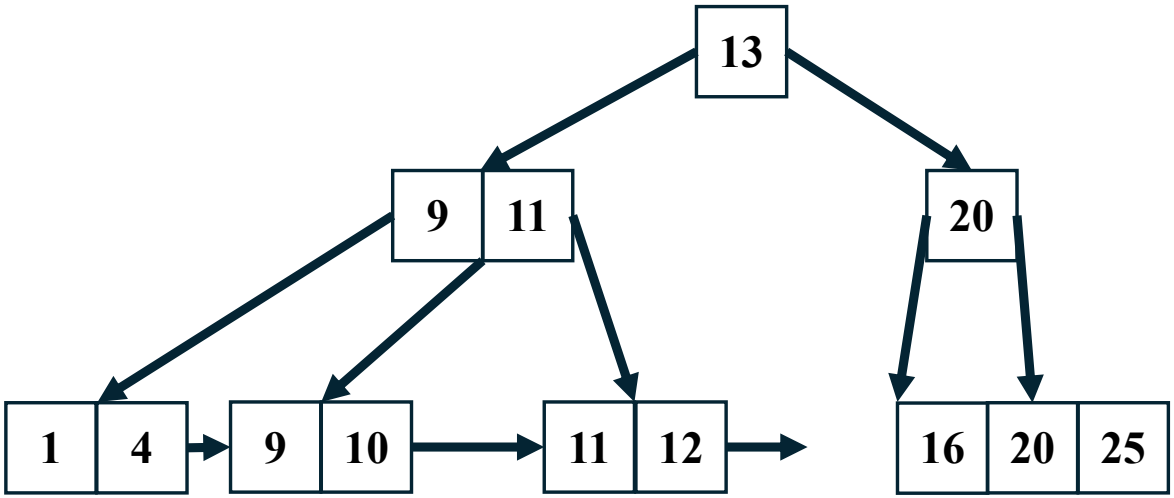
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Delete(15)



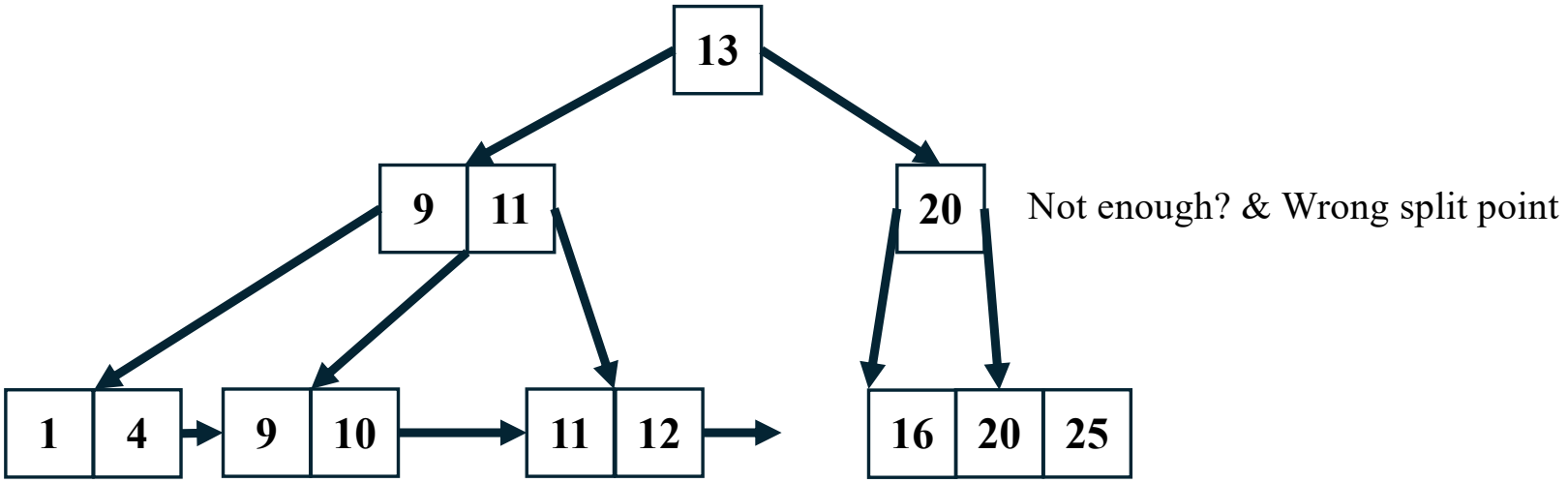
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Delete(15)



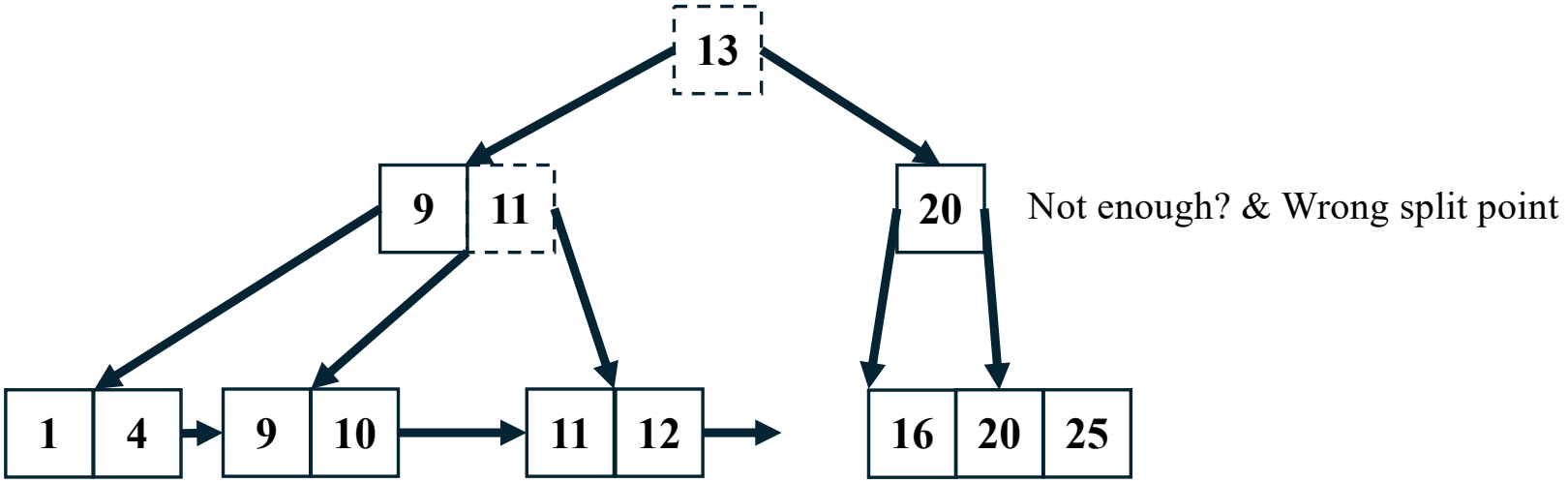
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Delete(15)



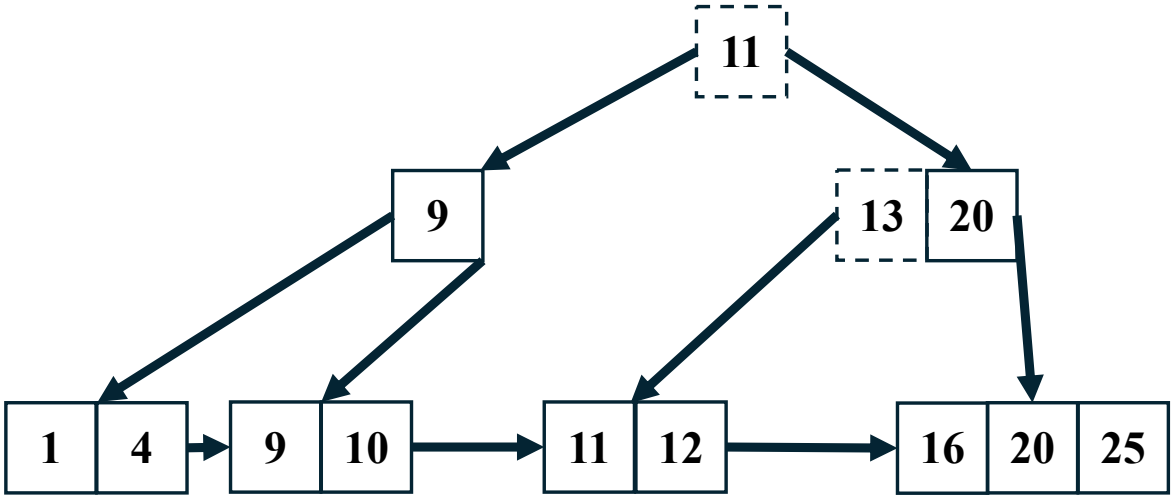
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Delete(15)



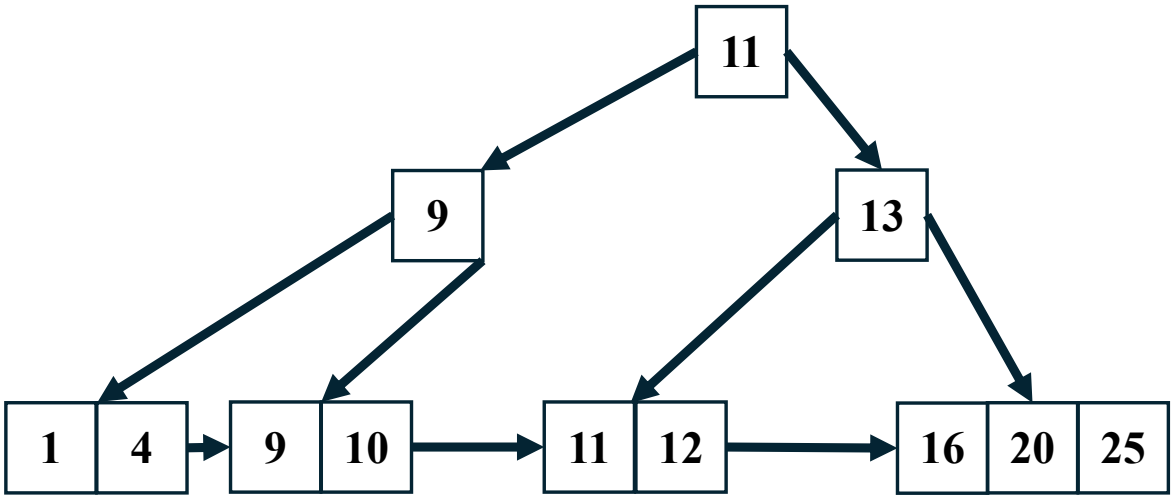
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Delete(15)



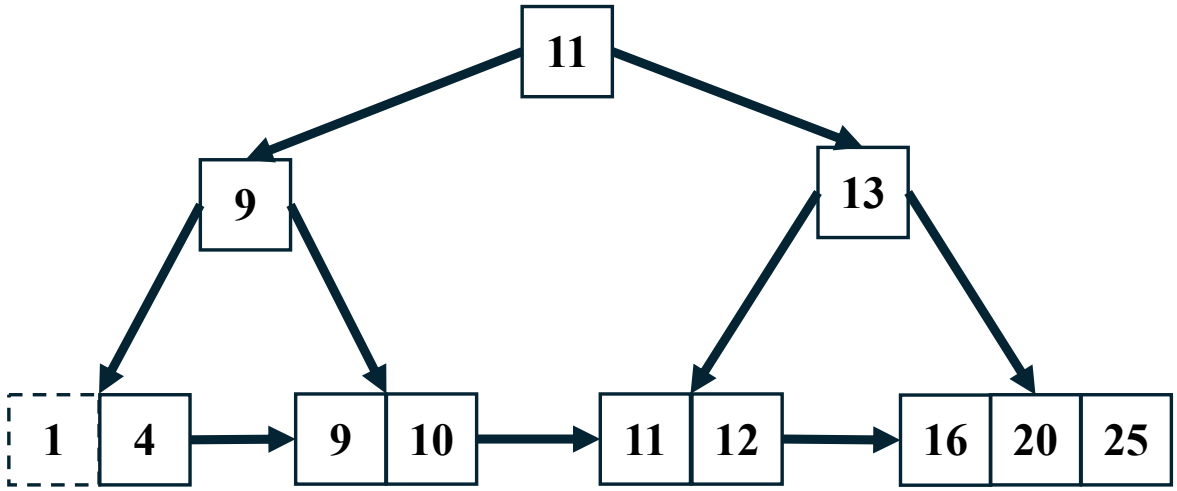
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Delete(15)



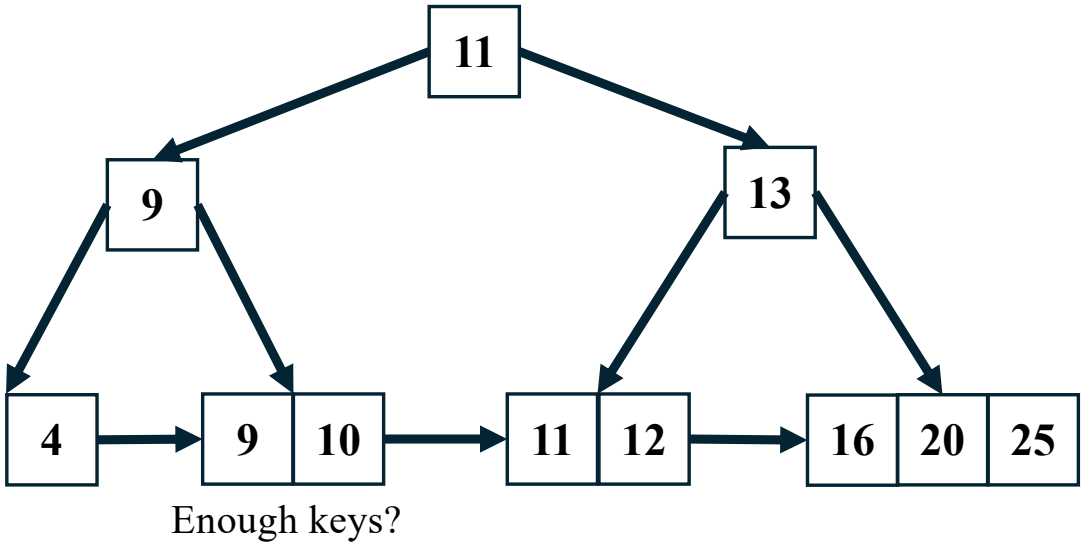
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Delete(1)



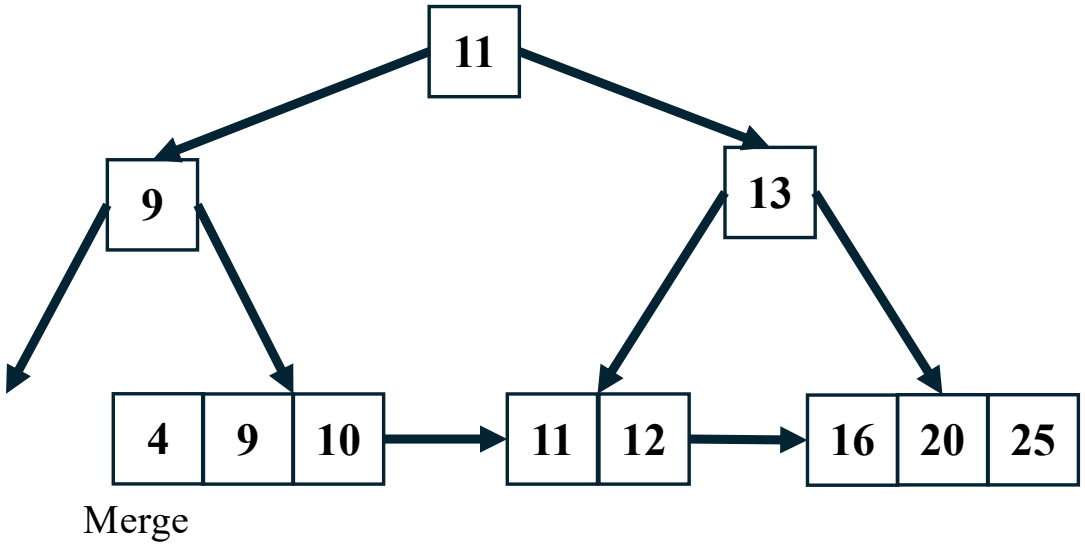
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Delete(1)



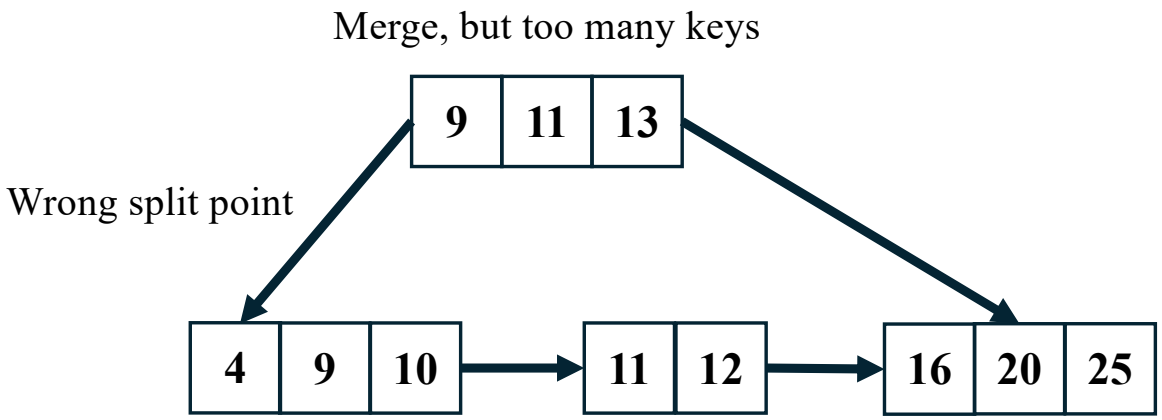
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Delete(1)



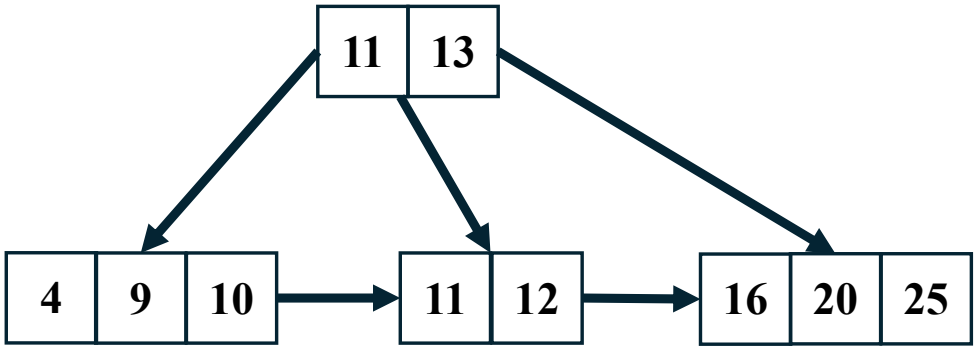
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Delete(1)



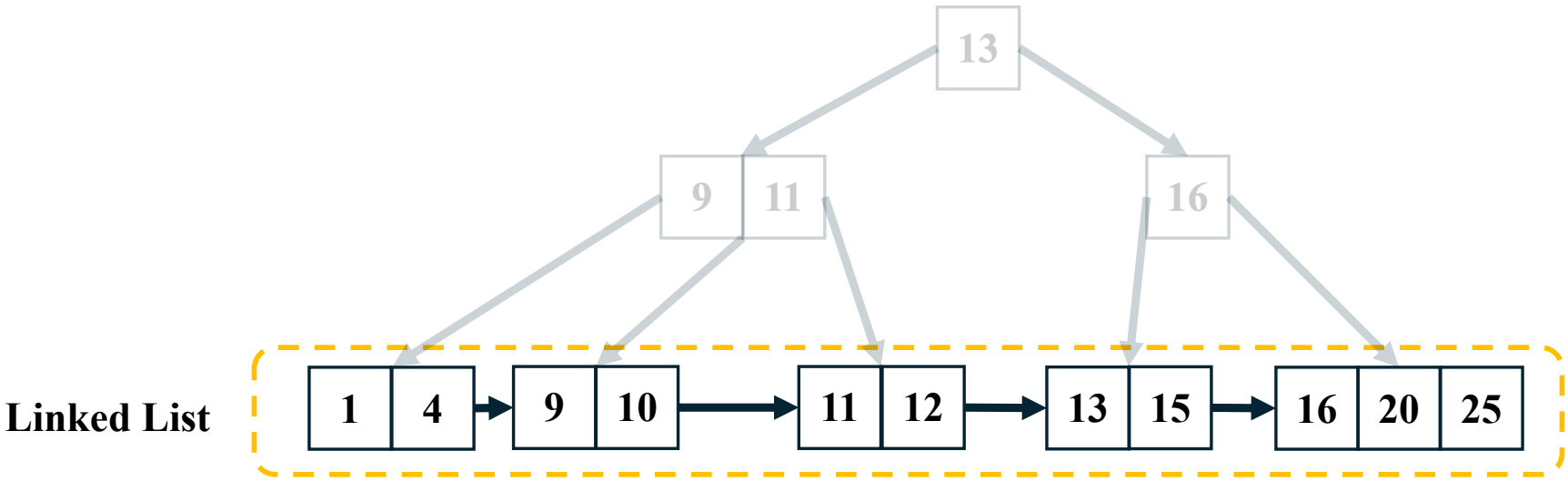
Bonus: B+Tree implementation

- B+Tree Example (M = 4): Delete(1)



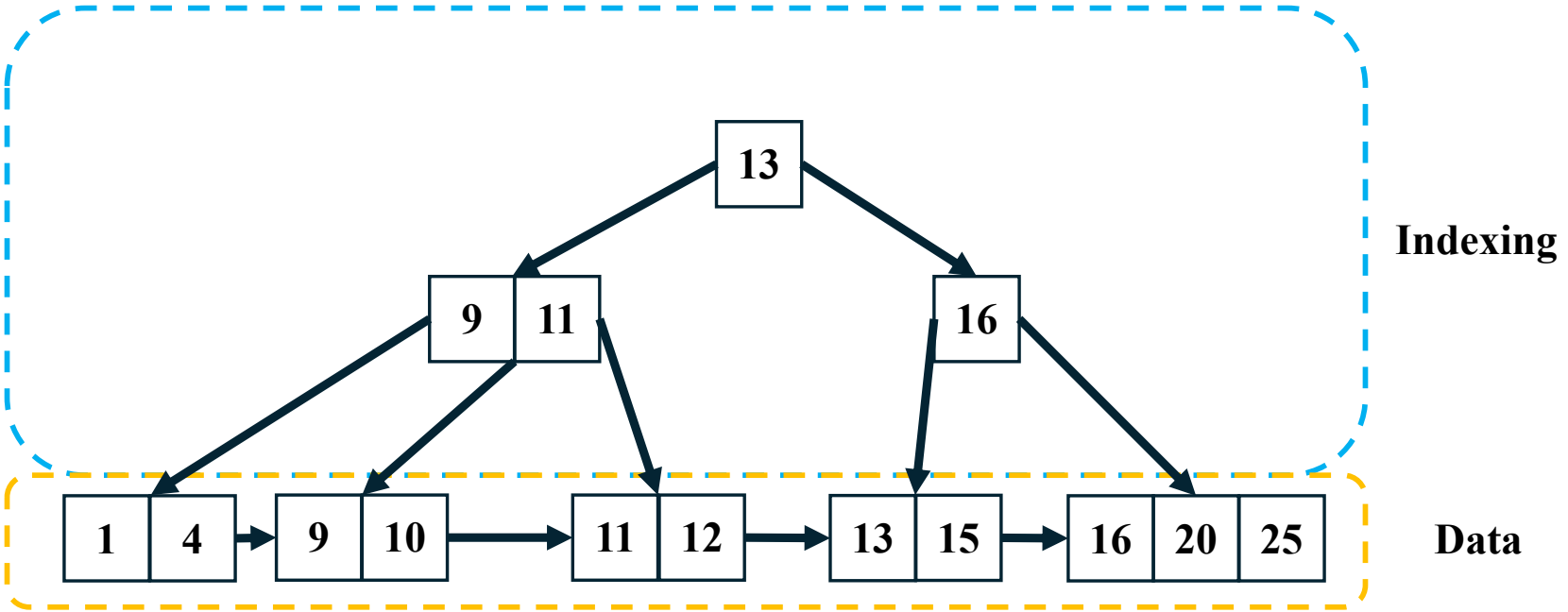
Bonus: B+Tree implementation

- In a B+tree, all the data records are stored in the leaf nodes, while the internal nodes only contain keys to guide the search process



Bonus: B+Tree implementation

- In a B+tree, all the data records are stored in the leaf nodes, while the internal nodes only contain keys to guide the search process



Bonus: B+Tree implementation

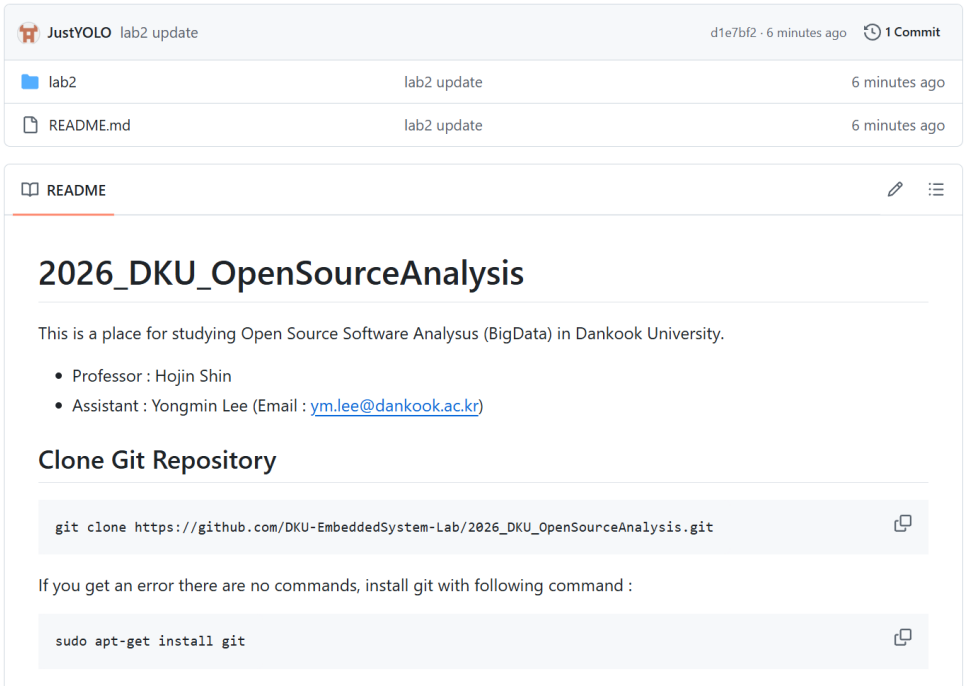
- Code

```cpp
#include "bptree.h"

#include <algorithm>

// B+Tree Constructor. degree 설정 등 최초 설정 진행
BPlusTree::BPlusTree(int degree) : root_(nullptr), degree_(degree) {
  // code
}

// B+Tree Destructor. B+Tree에 존재하는 모든 Node delete 필요
BPlusTree::~BPlusTree() {
  // code
}

// B+Tree Put operation
void BPlusTree::Put(int key, const std::string& value) {
  // code
}

// B+Tree Get operation
bool BPlusTree::Get(int key, std::string* value) const {
  // code
  return false;
}
```

```cpp
// B+Tree Range Scan operation
std::vector<std::pair<int, std::string>>
BPlusTree::RangeScan(int start_key, int end_key) const {
  std::vector<std::pair<int, std::string>> out;

  // code

  return out;
}

// B+Tree Delete operation. In-place update로 진행 됨으로, 실제 노드 삭제가
// 진행되야함
bool BPlusTree::Delete(int key) {

  // code

  return false;
}
```

Code Structure

- **Github repo**
  - https://github.com/DKU-EmbeddedSystem-Lab/2026_DKU_OpenSourceAnalysis



JustYOLO lab2 update — d1e7bf2 · 6 minutes ago — 1 Commit

lab2	lab2 update	6 minutes ago
README.md	lab2 update	6 minutes ago

README

2026_DKU_OpenSourceAnalysis

This is a place for studying Open Source Software Analysus (BigData) in Dankook University.

- Professor : Hojin Shin
- Assistant : Yongmin Lee (Email : ym.lee@dankook.ac.kr)

Clone Git Repository

```
git clone https://github.com/DKU-EmbeddedSystem-Lab/2026_DKU_OpenSourceAnalysis.git
```

If you get an error there are no commands, install git with following command :

```
sudo apt-get install git
```

Lab2: Skip list

If you want to proceed to Lab2 skip list, go to command below :

```
cd 2026_DKU_OpenSourceAnalysis/lab2/skiplist

make

./memdb_test
```

Lab2: B+Tree

If you want to proceed to Lab2 skip list, go to command below :

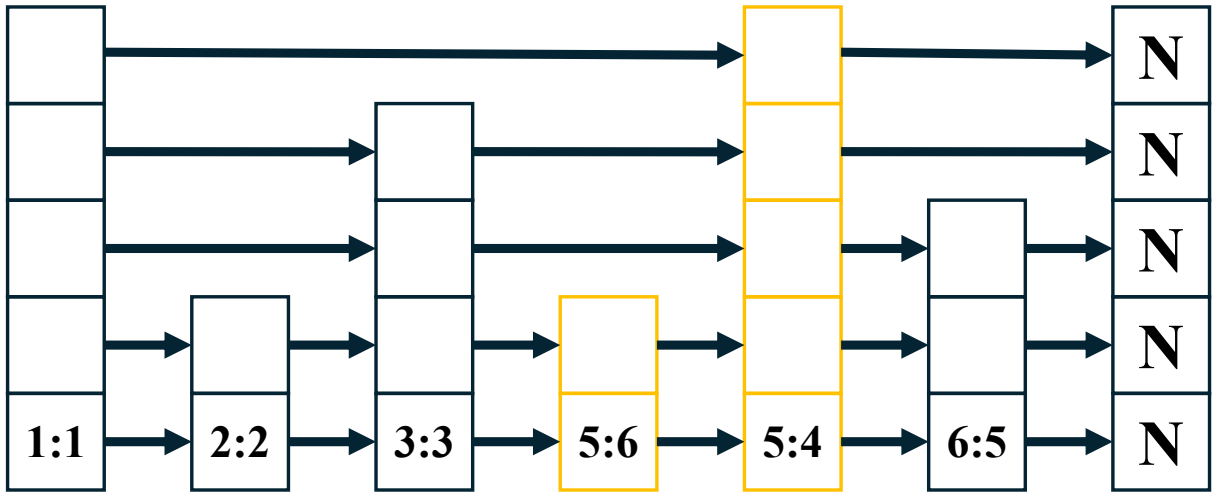
```
cd 2026_DKU_OpenSourceAnalysis/lab2/bptree

make

./memdb_test
```

Code Structure

Lab 2
- bptree
  - Makefile (DO NOT MODIFY!)
  - memdb_bench.cc (DO NOT MODIFY!)
  - memdb_test.cc (DO NOT MODIFY!)
  - **bptree.cc**
  - **bptree.h**
  - **memdb.cc**
  - **memdb.h**
- skiplist
  - Makefile (DO NOT MODIFY!)
  - memdb_bench.cc (DO NOT MODIFY!)
  - memdb_test.cc (DO NOT MODIFY!)
  - **skiplist.cc**
  - **skiplist.h**
  - **memdb.cc**
  - **memdb.h**
- common.mk (DO NOT MODIFY!)
- workloads (DO NOT MODIFY!)
  - …

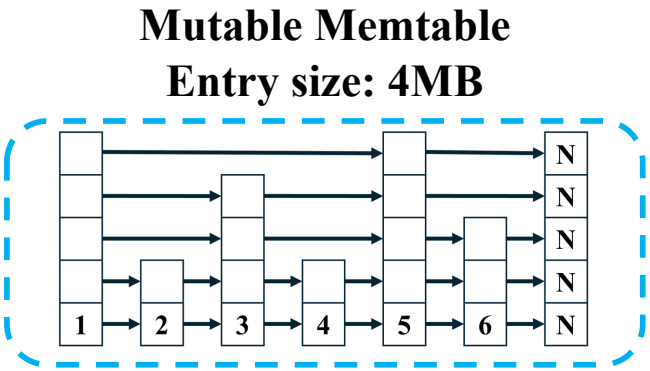
Code Structure

- **Skip list**
  - In-memory DB
    - Out-of-place update 사용 (delete도 tombstone 사용)
    - Sequence number(seq #)를 사용하여 key 구분
      - 들어온 순서대로 seq # 부여; 처음 들어온 것 부터 1, 이후 2, 3 …
      - Seq #가 큰 key가 가장 최신 값 (key : seq#)



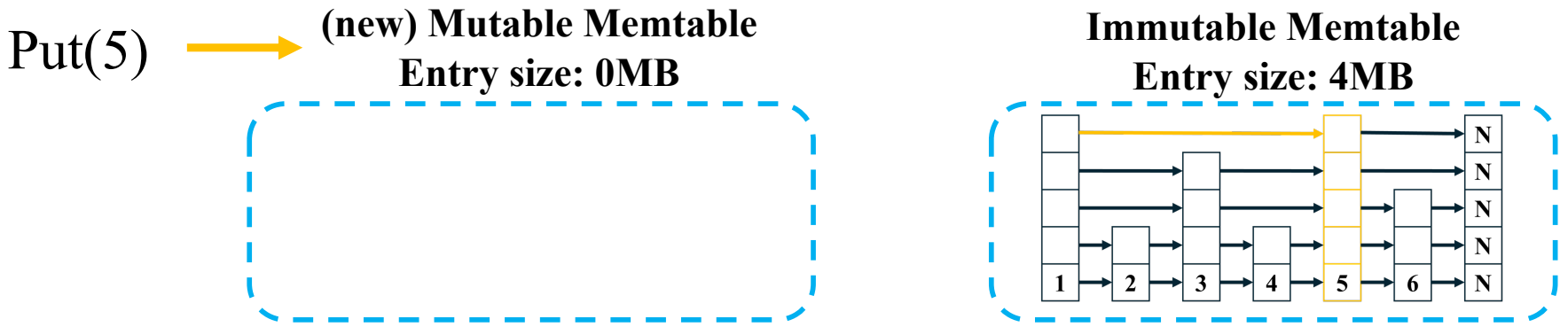
Code Structure

- **Skip list**
  - In-memory DB
    - 각 Memtable (Skip list)의 크기를 확인하여 mutable → immutable 변경
      - MemDBOptions.max_memtable_bytes 옵션으로 지정되어 있음 (기본 4MB, 변경 가능)



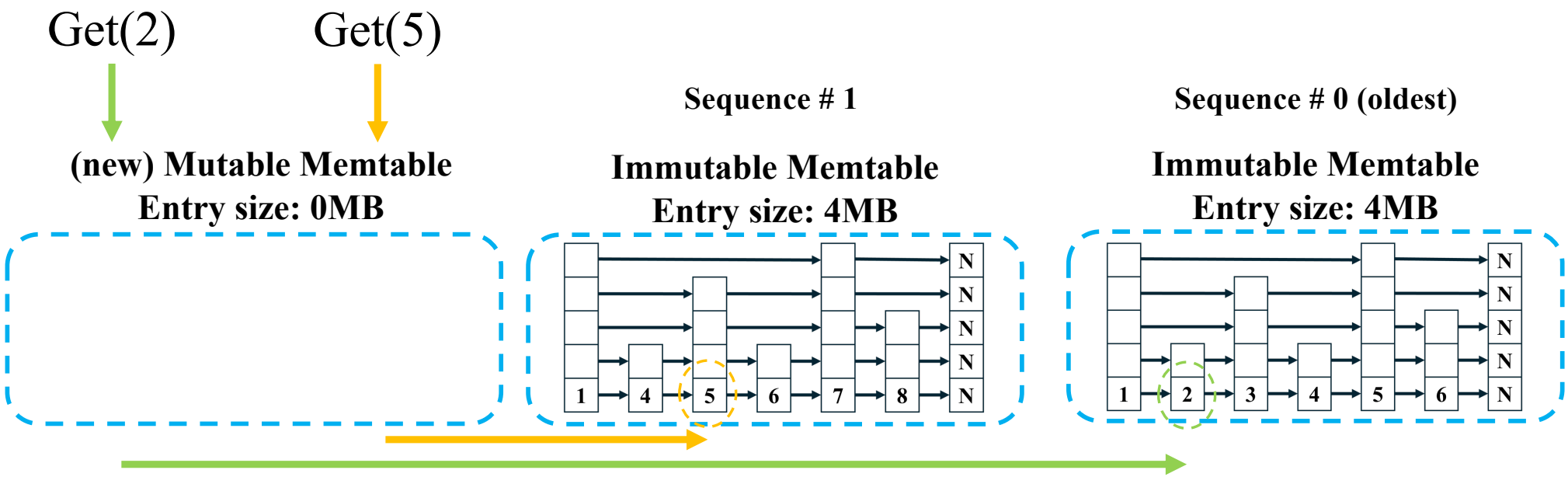
Code Structure

- **Skip list**
  - In-memory DB
    - 각 Memtable (Skip list)의 크기를 확인하여 mutable → immutable 변경
      - MemDBOptions.max_memtable_bytes 옵션으로 지정되어 있음 (기본 4MB, 변경 가능)



Code Structure

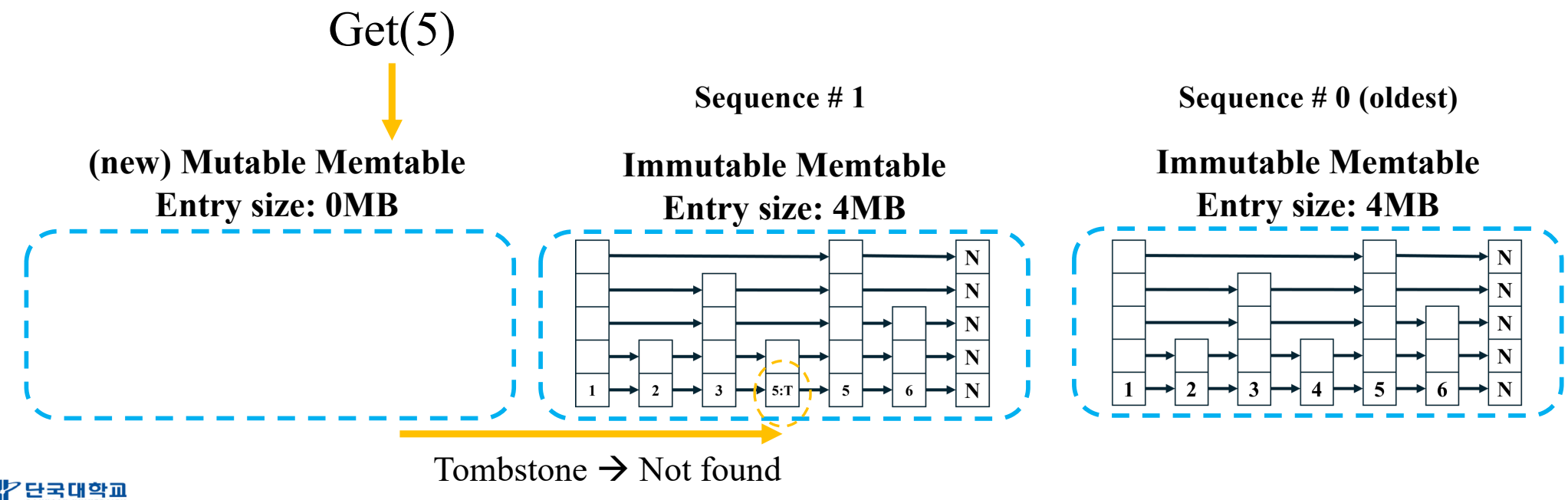
- **Skip list**
  - In-memory DB
    - Key search 시 모든 Memtable 검사 (가장 최근 memtable 순으로)



Code Structure

- **Skip list**
  - In-memory DB
    - Key search 시 모든 Memtable 검사 (가장 최근 memtable 순으로)
    - Tombstone 고려! (Range scan도 마찬가지)

      Tombstone발견시 not found 반환



Code Structure

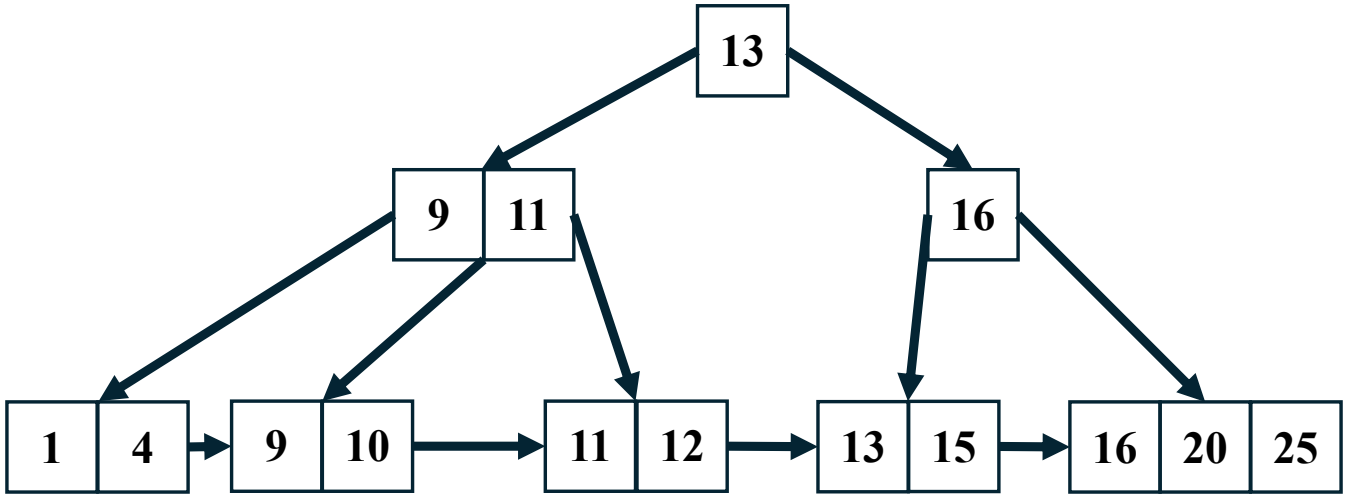
- **Skip list** put시 가득차면 imuutable로
  - In-memory DB code:

```cpp
#include "memdb.h"

#include <map>
#include <unordered_set>
#include <utility>

// SkipList를 사용하여 Out-of-place update를 진행하는 InMemoryDB
// Memtable 크기를 지정하여 가득 찼을 시 Immutable Memtable로 변경

InMemoryDB::MemTable::MemTable(const MemDBOptions& options)
    : list(options.skiplist_max_height, options.skiplist_p), size_bytes(0),
      immutable(false) {}

InMemoryDB::InMemoryDB(const MemDBOptions& options)
    : options_(options), mutable_(std::make_unique<MemTable>(options_)) {}

// Put operation 구현
void InMemoryDB::Put(int key, const std::string& value) {
  // code
}

// Get operation 구현
bool InMemoryDB::Get(int key, std::string* out_value) const {

  // code

  return false;
}
```

```cpp
// Delete operation 구현. Tombstone 사용
void InMemoryDB::Delete(int key) {
  // code
}

// RangeScan operation 구현
std::vector<std::pair<int, std::string>>
InMemoryDB::RangeScan(int start_key, int end_key) const {
  std::vector<std::pair<int, std::string>> out;

  // code

  return out;
}

// Memtable size 제한 확인하는 함수
void InMemoryDB::EnsureMutableCapacity(size_t entry_bytes) {
  // code
}
```

Code Structure

- **B+Tree**
  - In-memory DB:
    - In-place update 사용 (delete가 실제 노드 삭제로 이어져야 함)
    - 하나의 큰 B+Tree 사용



Code Structure

- **B+Tree**
  - In-memory DB code:

```cpp
#include "memdb.h"

InMemoryDB::InMemoryDB(const MemDBOptions& options)
    : tree_(options.bptree_degree) {}

void InMemoryDB::Put(int key, const std::string& value) {
  tree_.Put(key, value);
}

bool InMemoryDB::Get(int key, std::string* out_value) const {
  return tree_.Get(key, out_value);
}

bool InMemoryDB::Delete(int key) {
  return tree_.Delete(key);
}

std::vector<std::pair<int, std::string>> InMemoryDB::RangeScan(
    int start_key, int end_key) const {
  return tree_.RangeScan(start_key, end_key);
}
```

Code Structure

- **구현해야 하는 내용 정리**
  - **Skip list**
    - Skip list 내부 operation (초기화, put, get, 확률적 높이 설정, delete, range scan)
      - Range scan: 시작 key와 마지막 key 사이 모든 key를 반환
    - **memdb: 모든 Memtable 검색**
      - Put: Mutable Memtable(Skip list)에 key-value 삽입
      - Get: 가장 최신의 key에 대한 value 반환
      - Delete: Mutable Memtable(Skip list)에 key-tombstone 삽입
        - Tombstone 발견시 not found로 반환해야함 (tombstone을 value로 반환하면 안됨!)
      - Range Scan: 모든 Memtable을 검사해서 주어진 범위 내 key-value 반환
        - 주의! Mutable Memtable에 대한 range scan 결과 반환이 아님. Mutable, immutable Memtable을 모두 검사해서 유효한 value를 반환해야함
      - Memtable 관리: Mutable Memtable size를 확인해서 immutable로 변환

Code Structure

- 구현해야하는 내용 정리
  - **B+Tree**
    - B+Tree 내부 operation (초기화, put, get, delete, range scan)
    - 내부 operation에서 발생하는 balancing 과정 주의!
  - **memdb: 구현 필요한 부분 없음**
    - B+Tree는 in-place update를 사용
    - 하나의 큰 B+Tree만을 사용 (Memtable 개념 없음)

How to Submit

- **제출 파일: 코드**
  - 총 4개의 파일
    - **SkipList**
      - memdb.cc
      - memdb.h
      - skiplist.cc
      - skiplist.h
    - $ tar -cvf 학번_분반_skiplist_이름.tar memdb.cc memdb.h skiplist.cc skiplist.h
    - E.g., $ tar -cvf 32203333_1분반_skiplist_이용민.tar memdb.cc memdb.h skiplist.cc skiplist.h

    - **Bonus: B+Tree (필수 사항 아님!!)**
      - memdb.cc
      - memdb.h
      - bptree.cc
      - bptree.h
    - $ tar -cvf 학번_분반 _bptree_이름.tar memdb.cc memdb.h bptree.cc bptree.h
    - E.g., $ tar -cvf 32203333_1분반 _bptree_이용민.tar memdb.cc memdb.h bptree.cc bptree.h

    - **파일 추가 불가능! Makefile 등 이외 파일 수정 불가능!**

How to Submit

- **코드 제출시 주의사항**

  - **코드 자체에 자세한 주석 달기**
    - **코드 점수에 포함되어 있음**

  - Makefile, memdb_bench.cc, memdb_test.cc 등 수정 불가 파일 포함 시
→ 형식 미 준수로 감점 가능성
→ 코드 파일 archive 시 꼭 상기한 4개의 파일만 포함

  - Makefile, memdb_bench.cc, memdb_test.cc 등을 수정 시
→ 반영 안됨. 기본 상태의 Makefile, 코드 파일로 컴파일 및 채점 예정
→ 이로 인한 컴파일 불가, test fail, runtime error 발생 시 감점
→ 수정 가능한 파일만 수정해서 구현

How to Submit

- **코드 작성 시 주의사항**
  - 코드 채점은 다음과 같은 VM 환경에서 진행 될 예정임:
    - Host CPU: i7-12700K (AMD64)
    - vCPU: host, 4 cores (qemu-system-x86_64)
    - OS: Ubuntu 22.04.5 LTS
    - Linux Kernel: 5.15.0-173-generic
    - G++: 11.4.0 (C++17)
    - Make: GNU Make 4.3
    - **!! 꼭 환경을 맞출 필요는 없음 !!**
  - GCC 이외 compiler 사용시 non-standard function / header 주의
    - MSVC (Visual Studio) 등 사용시 주의
    - 감점 요인 및 미제출로 간주될 수 있음
  - C++ Standard Library (STL) 이외 external library 포함 시 compile error가 발생할 수 있음
    - 이 경우, 코드 미제출로 간주됨

How to Submit

- **제출 파일**
  - **코드를 묶은 .tar 파일**
    - (학번_분반 _(skiplist 혹은 bptree)_이름.tar) (e.g., 32203333_1분반_skiplist_이용민.tar)

  → 이외 .egg 등 사용시 감점 혹은 미제출로 간주

  - **보고서 (PDF 파일로 제출)**
    - 코드 설명:
      - Put, Get 등 기본적으로 작성해야 하는 코드 설명
      - 추가로 생성한 함수 등도 자세하게 설명
    - 고찰(Discussion)
      - 고찰은 최소한 하나의 실험 결과와 해당하는 그래프, 실험 결과에 대한 분석 필요
      - 실험은 최소한 하나의 변인(e.g., MemTable size, skip list height, B+Tree의 M값 등)을 지정하여 결과를 도출해야함
  - **보고서 제목:** lab2_32XXXXXX_분반_이름**.pdf**
    - E.g., lab2_ 32203333_1분반_이용민**.pdf**

PDF만 제출 가능! 이외 형식으로 제출시 미제출로 간주됩니다.
제출 형식을 꼭 지켜주세요. 감점 요인입니다.

  - 제출 방법: Google Form
    - https://docs.google.com/forms/d/e/1FAIpQLSd9xPOCrGao99leQWY9mxhqWEVhDDdDjwnT_YiMtYm5fWR9Zg/viewform?usp=publish-editor

  - 제출 기한: ~2026.04.14 (화) 23시 59분까지

How to Submit

- **보고서 내용 및 형식 주의 사항**
  - **표지에 제목, 과목 명, 분반, 소속 학과, 학번, 이름, 제출일 작성 (미 작성시 감점)**
  - **LLM 출력 단순 캡쳐, 글을 사진 파일로 붙이기, PDF 외 형식으로 제출은 미제출로 간주될 수 있음**
  - LLM 사용 가능. 하지만 틀린 내용, 주제와 무관한 내용 작성시 감점
    - 참고한 내용은 참고 문헌 작성 필수 (RocksDB Wiki 포함)
  - 표절 검사 진행하여 최고 표절율이 30% 이상일 시 소명 필요
    - 추후 감점 혹은 미제출로 간주될 수 있음
  - 이외 중요 내용 누락, 표절 의심, 지각 제출 등 감점

Appendix. How to run

- **Make**

```
lee@lee-vm:~/2026_DKU_OpenSourceAnalysis/lab2/.answer/skiplist$ l
total 52K
drwxrwxr-x 2 lee lee 4.0K Mar 27 08:30 .
drwxrwxr-x 6 lee lee 4.0K Mar 26 08:15 ..
-rw-rw-r-- 1 lee lee  587 Mar 26 05:47 Makefile
-rw-rw-r-- 1 lee lee 2.5K Mar 26 05:47 memdb.cc
-rw-rw-r-- 1 lee lee 1.1K Mar 26 05:47 memdb.h
-rw-rw-r-- 1 lee lee 8.3K Mar 26 05:47 memdb_bench.cc
-rw-rw-r-- 1 lee lee 7.5K Mar 26 05:47 memdb_test.cc
-rw-rw-r-- 1 lee lee 4.4K Mar 26 05:47 skiplist.cc
-rw-rw-r-- 1 lee lee 1.2K Mar 26 05:47 skiplist.h
lee@lee-vm:~/2026_DKU_OpenSourceAnalysis/lab2/.answer/skiplist$ make
g++ -std=c++17 -O2 -Wall -Wextra -pthread -I../googletest/include -I../googletest -c memdb_test.cc -o memdb_test.o
g++ -std=c++17 -O2 -Wall -Wextra -pthread -I../googletest/include -I../googletest -c memdb.cc -o memdb.o
g++ -std=c++17 -O2 -Wall -Wextra -pthread -I../googletest/include -I../googletest -c skiplist.cc -o skiplist.o
g++ -std=c++17 -O2 -Wall -Wextra -pthread -I../googletest/include -I../googletest -c ../googletest/src/gtest-all.cc -o gtest-all.o
g++ -std=c++17 -O2 -Wall -Wextra -pthread -o memdb_test memdb_test.o memdb.o skiplist.o gtest-all.o -lpthread
g++ -std=c++17 -O2 -Wall -Wextra -pthread -I../googletest/include -I../googletest -c memdb_bench.cc -o memdb_bench.o
g++ -std=c++17 -O2 -Wall -Wextra -pthread -o memdb_bench memdb_bench.o memdb.o skiplist.o
lee@lee-vm:~/2026_DKU_OpenSourceAnalysis/lab2/.answer/skiplist$ l
total 1.8M
drwxrwxr-x 2 lee lee 4.0K Mar 27 08:31 .
drwxrwxr-x 6 lee lee 4.0K Mar 26 08:15 ..
-rw-rw-r-- 1 lee lee  587 Mar 26 05:47 Makefile
-rw-rw-r-- 1 lee lee 860K Mar 27 08:31 gtest-all.o
-rw-rw-r-- 1 lee lee 2.5K Mar 26 05:47 memdb.cc
-rw-rw-r-- 1 lee lee 1.1K Mar 26 05:47 memdb.h
-rw-rw-r-- 1 lee lee  19K Mar 27 08:31 memdb.o
-rwxrwxr-x 1 lee lee  51K Mar 27 08:31 memdb_bench
-rw-rw-r-- 1 lee lee 8.3K Mar 26 05:47 memdb_bench.cc
-rw-rw-r-- 1 lee lee  26K Mar 27 08:31 memdb_bench.o
-rwxrwxr-x 1 lee lee 597K Mar 27 08:31 memdb_test
-rw-rw-r-- 1 lee lee 7.5K Mar 26 05:47 memdb_test.cc
-rw-rw-r-- 1 lee lee 184K Mar 27 08:31 memdb_test.o
-rw-rw-r-- 1 lee lee 4.4K Mar 26 05:47 skiplist.cc
-rw-rw-r-- 1 lee lee 1.2K Mar 26 05:47 skiplist.h
-rw-rw-r-- 1 lee lee  18K Mar 27 08:31 skiplist.o
lee@lee-vm:~/2026_DKU_OpenSourceAnalysis/lab2/.answer/skiplist$
```

Appendix. How to run

- **Testing**

```
lee@lee-vm:~/2026_DKU_OpenSourceAnalysis/lab2/skiplist$ ./memdb_test
[==========] Running 11 tests from 2 test suites.
[----------] Global test environment set-up.
[----------] 6 tests from MemDBTest
[ RUN      ] MemDBTest.PutGetBasic
memdb_test.cc:84: Failure
Value of: db.Get(1, &value)
  Actual: false
Expected: true

memdb_test.cc:85: Failure
Expected equality of these values:
  value
    Which is: ""
  "one"

memdb_test.cc:86: Failure
Value of: db.Get(2, &value)
  Actual: false
Expected: true

memdb_test.cc:87: Failure
Expected equality of these values:
  value
    Which is: ""
  "two"

[  FAILED  ] MemDBTest.PutGetBasic (0 ms)
[ RUN      ] MemDBTest.DeleteMasksOlderValue
memdb_test.cc:102: Failure
Value of: db.Get(2, &value)
  Actual: false
Expected: true

memdb_test.cc:103: Failure
Expected equality of these values:
  value
    Which is: ""
  "v2"
```

Appendix. How to run

- **Testing**

```
[----------] Global test environment tear-down
[==========] 11 tests from 2 test suites ran. (1385 ms total)
[  PASSED  ] 0 tests.
[  FAILED  ] 11 tests, listed below:
[  FAILED  ] MemDBTest.PutGetBasic
[  FAILED  ] MemDBTest.DeleteMasksOlderValue
[  FAILED  ] MemDBTest.ImmutableSearchOrder
[  FAILED  ] MemDBTest.TombstoneStopsOlderLookup
[  FAILED  ] MemDBTest.MultipleVersionsSameKey
[  FAILED  ] MemDBTest.TombstoneMasksOlderVersions
[  FAILED  ] MemDBWorkloadTest.FillRandom
[  FAILED  ] MemDBWorkloadTest.FillSequential
[  FAILED  ] MemDBWorkloadTest.RangeScanAcrossMemtables
[  FAILED  ] MemDBWorkloadTest.FillRandom2
[  FAILED  ] MemDBWorkloadTest.FillDeleteRandom

11 FAILED TESTS
lee@lee-vm:~/2026_DKU_OpenSourceAnalysis/lab2/skiplist$
```

Appendix. How to run

- **Testing**

```
lee@lee-vm:~/2026_DKU_OpenSourceAnalysis/lab2/.answer/skiplist$ ./memdb_test
[==========] Running 11 tests from 2 test suites.
[----------] Global test environment set-up.
[----------] 6 tests from MemDBTest
[ RUN      ] MemDBTest.PutGetBasic
[       OK ] MemDBTest.PutGetBasic (0 ms)
[ RUN      ] MemDBTest.DeleteMasksOlderValue
[       OK ] MemDBTest.DeleteMasksOlderValue (0 ms)
[ RUN      ] MemDBTest.ImmutableSearchOrder
[       OK ] MemDBTest.ImmutableSearchOrder (0 ms)
[ RUN      ] MemDBTest.TombstoneStopsOlderLookup
[       OK ] MemDBTest.TombstoneStopsOlderLookup (0 ms)
[ RUN      ] MemDBTest.MultipleVersionsSameKey
[       OK ] MemDBTest.MultipleVersionsSameKey (0 ms)
[ RUN      ] MemDBTest.TombstoneMasksOlderVersions
[       OK ] MemDBTest.TombstoneMasksOlderVersions (0 ms)
[----------] 6 tests from MemDBTest (0 ms total)

[----------] 5 tests from MemDBWorkloadTest
[ RUN      ] MemDBWorkloadTest.FillRandom
[       OK ] MemDBWorkloadTest.FillRandom (2970 ms)
[ RUN      ] MemDBWorkloadTest.FillSequential
[       OK ] MemDBWorkloadTest.FillSequential (420 ms)
[ RUN      ] MemDBWorkloadTest.RangeScanAcrossMemtables
[       OK ] MemDBWorkloadTest.RangeScanAcrossMemtables (0 ms)
[ RUN      ] MemDBWorkloadTest.FillRandom2
[       OK ] MemDBWorkloadTest.FillRandom2 (614 ms)
[ RUN      ] MemDBWorkloadTest.FillDeleteRandom
[       OK ] MemDBWorkloadTest.FillDeleteRandom (805 ms)
[----------] 5 tests from MemDBWorkloadTest (4811 ms total)

[----------] Global test environment tear-down
[==========] 11 tests from 2 test suites ran. (4812 ms total)
[  PASSED  ] 11 tests.
lee@lee-vm:~/2026_DKU_OpenSourceAnalysis/lab2/.answer/skiplist$
```

Appendix. How to run

- **Testing**

```
lee@lee-vm:~/2026_DKU_OpenSourceAnalysis/lab2/.answer/bptree$ ./memdb_test
[==========] Running 9 tests from 2 test suites.
[----------] Global test environment set-up.
[----------] 4 tests from MemDBTest
[ RUN      ] MemDBTest.PutGetBasic
[       OK ] MemDBTest.PutGetBasic (0 ms)
[ RUN      ] MemDBTest.DeleteRemovesKey
[       OK ] MemDBTest.DeleteRemovesKey (0 ms)
[ RUN      ] MemDBTest.MultipleVersionsSameKey
[       OK ] MemDBTest.MultipleVersionsSameKey (0 ms)
[ RUN      ] MemDBTest.RangeScanBasic
[       OK ] MemDBTest.RangeScanBasic (0 ms)
[----------] 4 tests from MemDBTest (0 ms total)

[----------] 5 tests from MemDBWorkloadTest
[ RUN      ] MemDBWorkloadTest.FillRandom
[       OK ] MemDBWorkloadTest.FillRandom (2242 ms)
[ RUN      ] MemDBWorkloadTest.FillSequential
[       OK ] MemDBWorkloadTest.FillSequential (413 ms)
[ RUN      ] MemDBWorkloadTest.FillRandom2
[       OK ] MemDBWorkloadTest.FillRandom2 (309 ms)
[ RUN      ] MemDBWorkloadTest.RangeScanFromWorkload
[       OK ] MemDBWorkloadTest.RangeScanFromWorkload (0 ms)
[ RUN      ] MemDBWorkloadTest.FillDeleteRandom
[       OK ] MemDBWorkloadTest.FillDeleteRandom (348 ms)
[----------] 5 tests from MemDBWorkloadTest (3314 ms total)

[----------] Global test environment tear-down
[==========] 9 tests from 2 test suites ran. (3314 ms total)
[  PASSED  ] 9 tests.
lee@lee-vm:~/2026_DKU_OpenSourceAnalysis/lab2/.answer/bptree$
```

Appendix. How to run

- **Benchmarking**

```
lee@lee-vm:~/2026_DKU_OpenSourceAnalysis/lab2/.answer/skiplist$ ./memdb_bench
memdb_bench options:
  --benchmarks=fillrandom|fillseq|readrandom|readseq|mixed|rangeseq|rangerand
  --num=NUM
  --key_range=NUM
  --value_size=NUM
  --range_size=NUM
  --memtable_bytes=NUM
  --skiplist_p=FLOAT
  --skiplist_height=NUM
  --seed=NUM
Example:
  ./memdb_bench --benchmarks=fillrandom --num=100000 --value_size=100
  ./memdb_bench --benchmarks=fillrandom,readrandom --num=100000
```

```
lee@lee-vm:~/2026_DKU_OpenSourceAnalysis/lab2/.answer/skiplist$ ./memdb_bench --benchmarks=fillrandom --num=100000 --value_size=100
benchmark=fillrandom num=100000 key_range=100000 value_size=100 range_size=100 memtable_bytes=4194304 skiplist_p=0.5 skiplist_height=16 elapsed_sec=0.0511375
lee@lee-vm:~/2026_DKU_OpenSourceAnalysis/lab2/.answer/skiplist$ ./memdb_bench --benchmarks=fillrandom,readrandom --num=100000
benchmark=fillrandom num=100000 key_range=100000 value_size=100 range_size=100 memtable_bytes=4194304 skiplist_p=0.5 skiplist_height=16 elapsed_sec=0.0365663
benchmark=readrandom num=100000 key_range=100000 value_size=100 range_size=100 memtable_bytes=4194304 skiplist_p=0.5 skiplist_height=16 elapsed_sec=0.0958209 found=63043 requested=100000
lee@lee-vm:~/2026_DKU_OpenSourceAnalysis/lab2/.answer/skiplist$
```

Discussion

- Email: ym.lee@dankook.ac.kr

Acknowledgment

- SW중심대학
  - 본 교재는 2026년도 과학기술정보통신부 및 정보통신기획평가원의 ‘SW중심대학 사업’ 지원을 받아 제작 되었습니다.
  - 본 결과물의 내용을 전재할 수 없으며, 인용(재사용)할 때에는 반드시 과학기술정보통신부와 정보통신기획평가원이 지원한 ‘SW중심대학’의 결과물이라는 출처를 밝혀야 합니다.

IITP 정보통신기획평가원
디지털인재양성단 SW인재팀